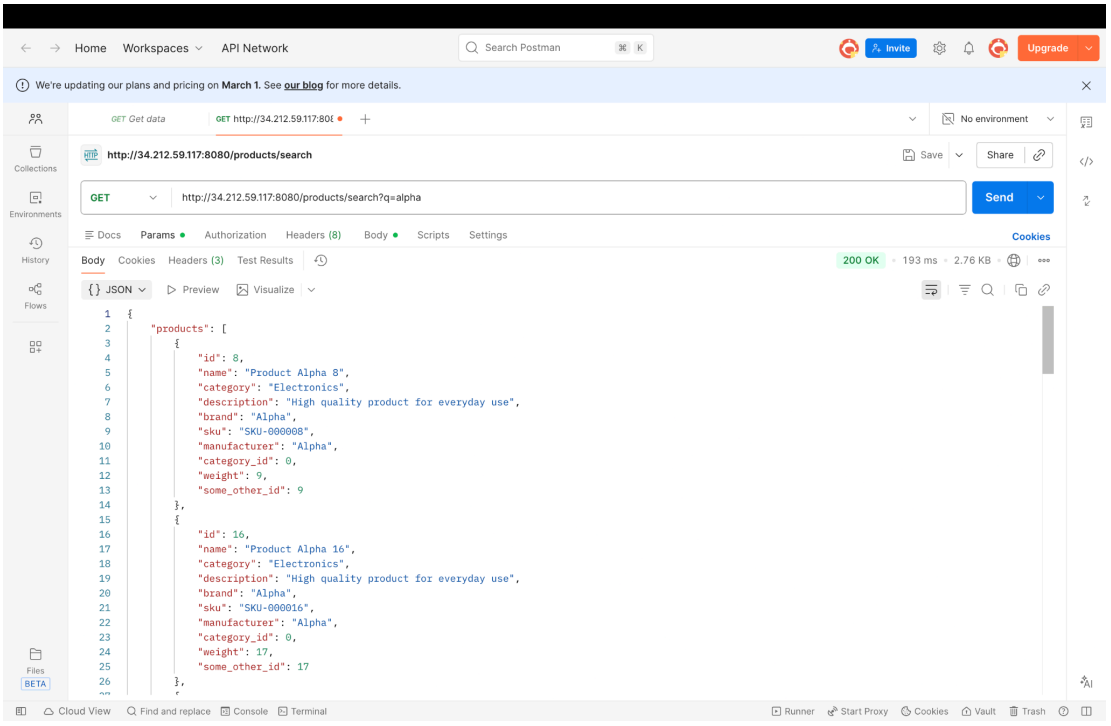
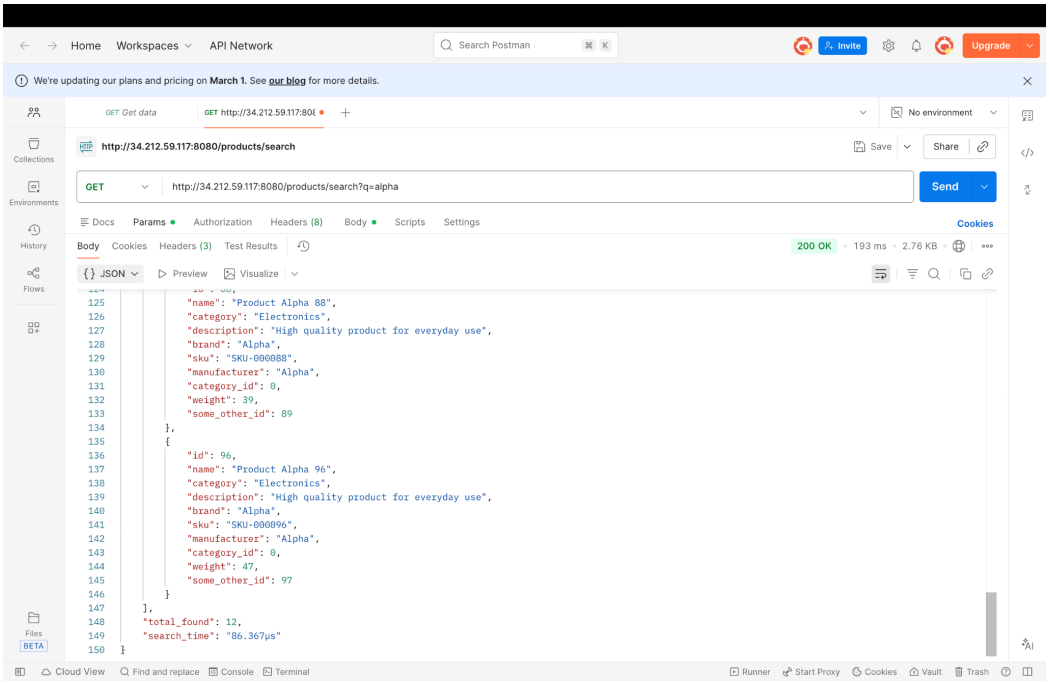
Report HW6

Part 2: Identifying Performance Bottlenecks

Single Request Verification

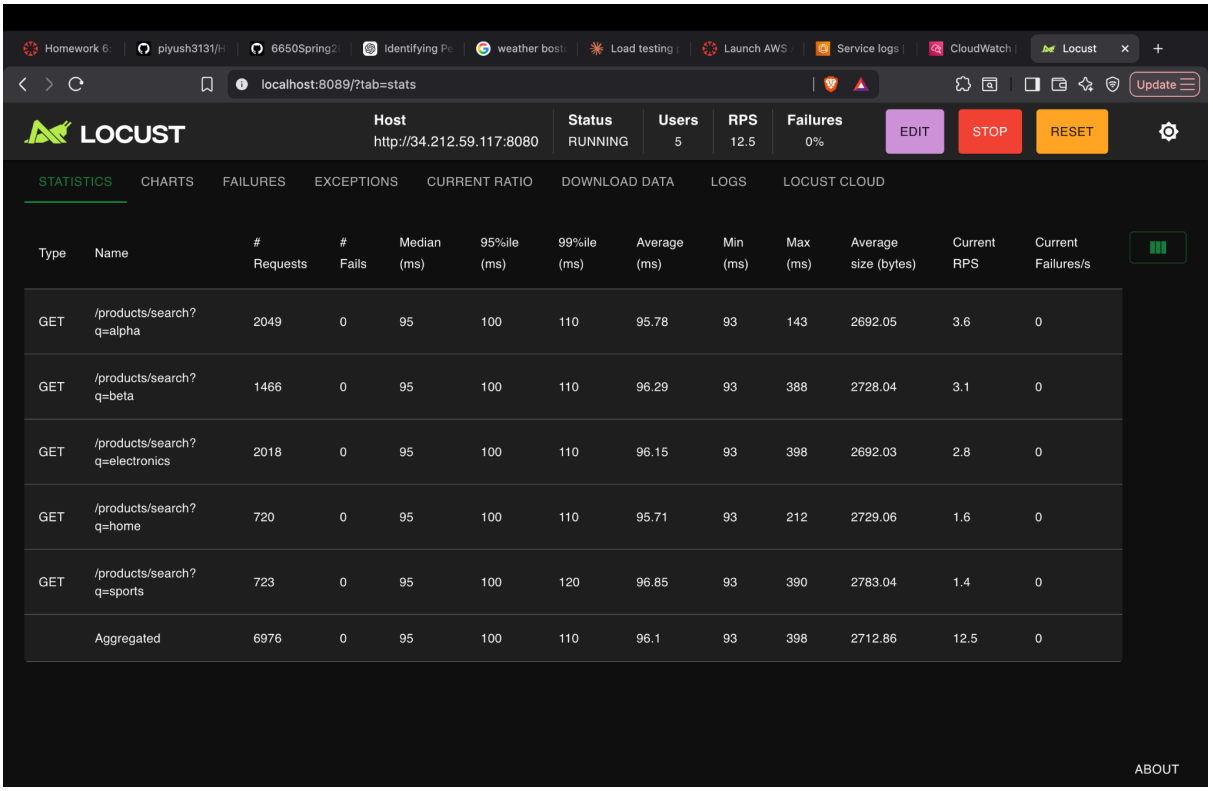
a single search was verified via curl. The search checked exactly 100 products, returned 12 matches for the query "alpha", and completed in **~90µs (0.09ms)** — under the 20ms target



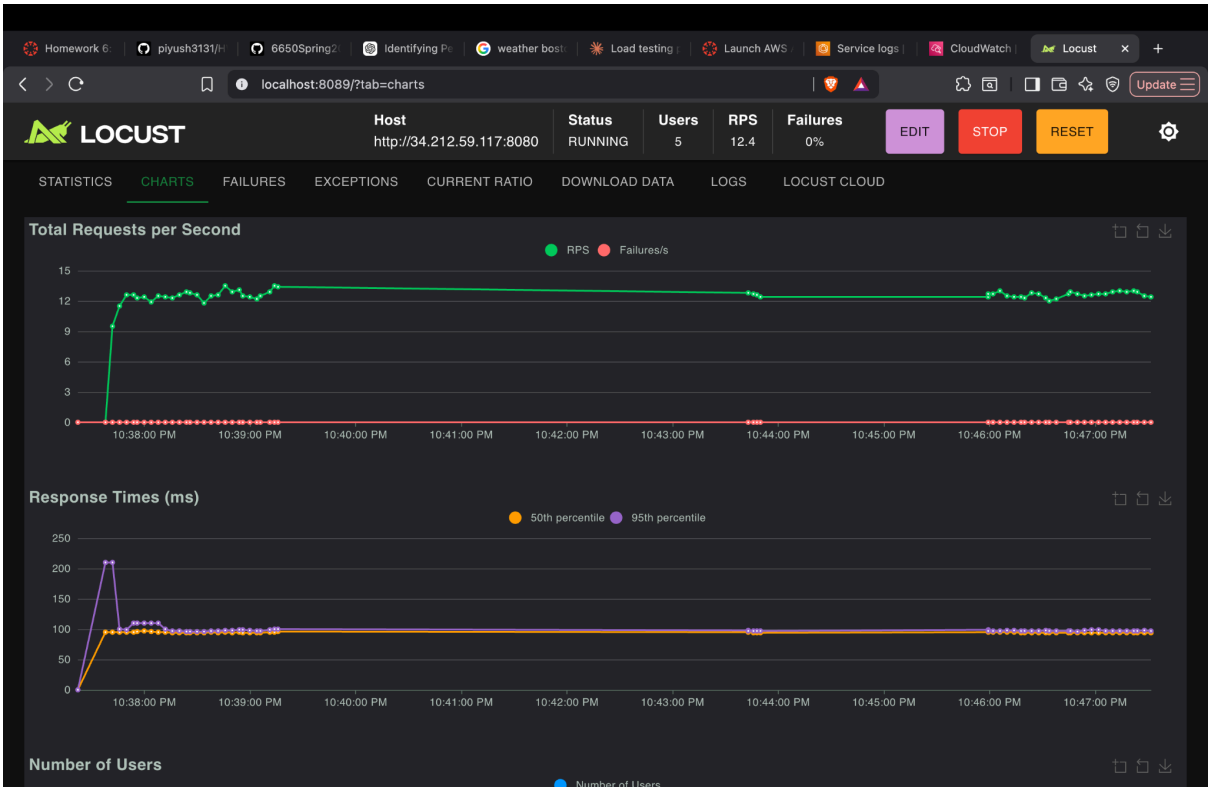


Load Increased

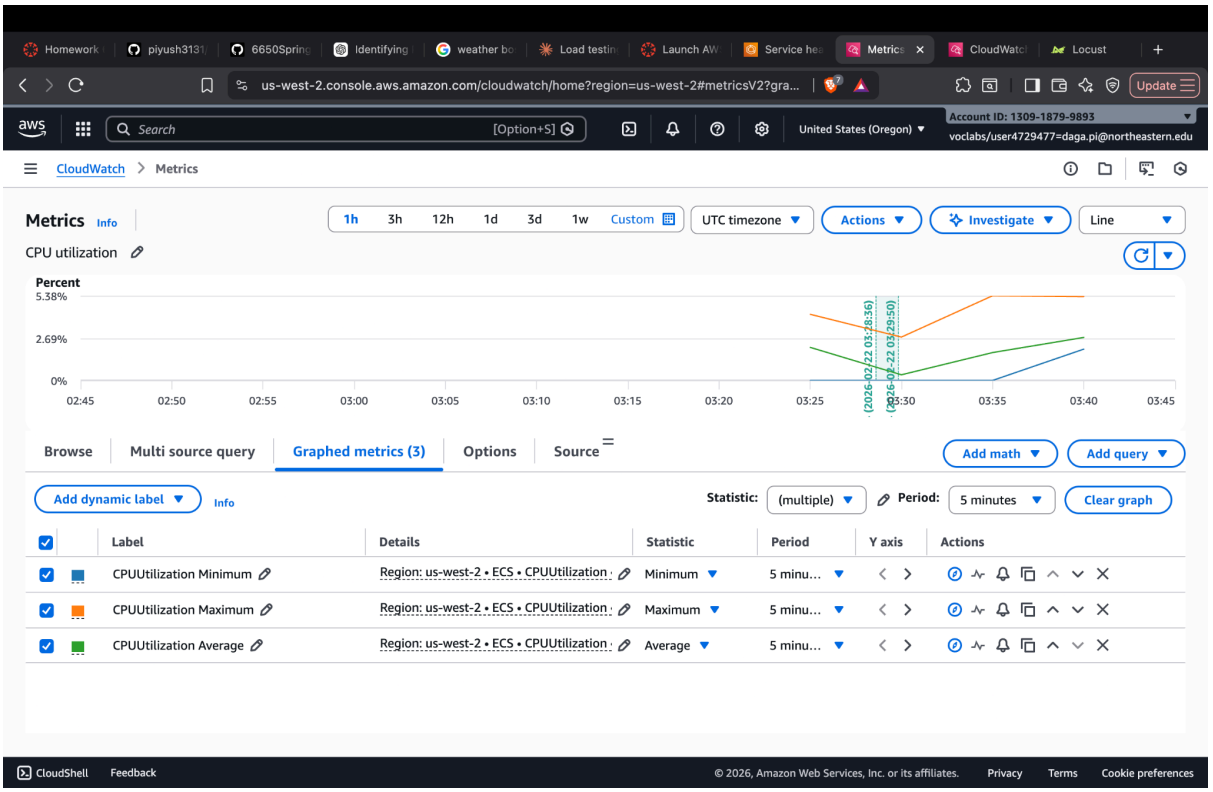
Test 1 — (5 users, 2 minutes): The service handled 5 concurrent users comfortably. Average response time was 96ms with consistent throughput of ~12.5 RPS. CPU utilization stayed around 5%, indicating plenty of room.



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/search?q=alpha	2049	0	95	100	110	95.78	93	143	2692.05	3.6	0
GET	/products/search?q=beta	1466	0	95	100	110	96.29	93	388	2728.04	3.1	0
GET	/products/search?q=electronics	2018	0	95	100	110	96.15	93	398	2692.03	2.8	0
GET	/products/search?q=home	720	0	95	100	110	95.71	93	212	2729.06	1.6	0
GET	/products/search?q=sports	723	0	95	100	120	96.85	93	390	2783.04	1.4	0
	Aggregated	6976	0	95	100	110	96.1	93	398	2712.86	12.5	0

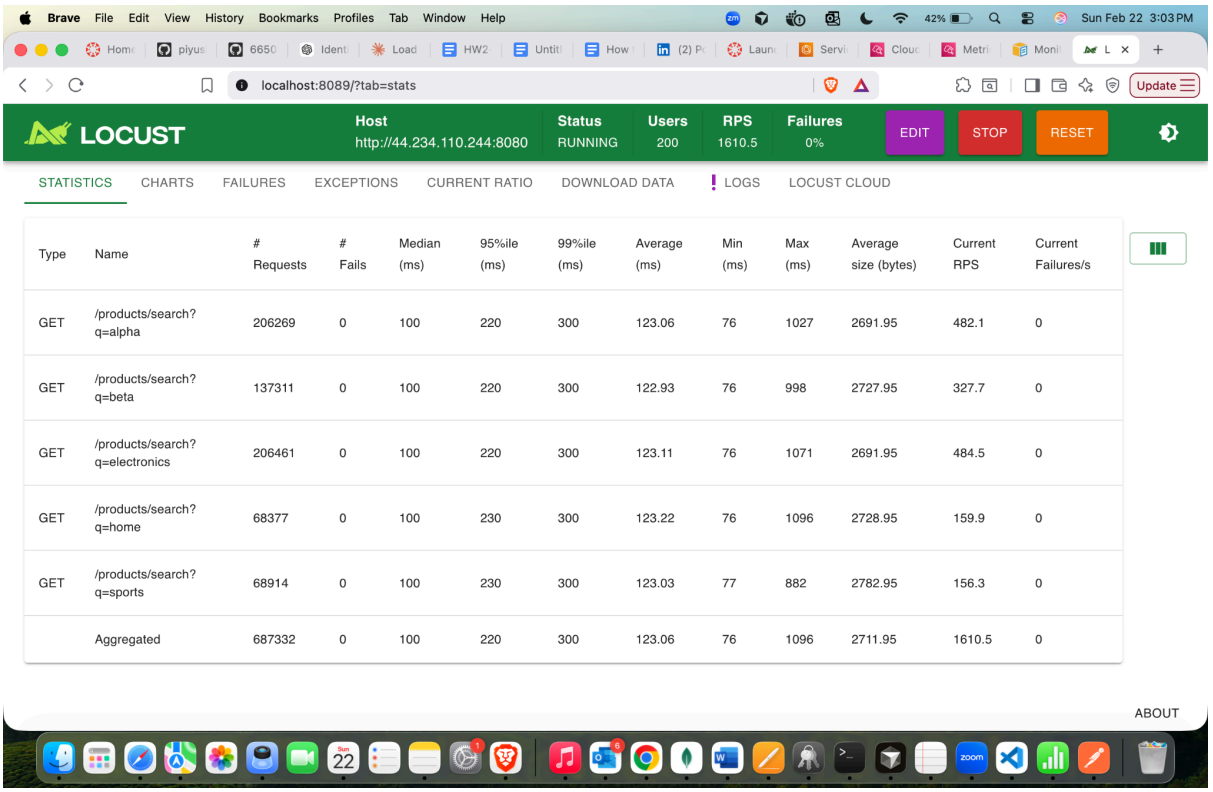


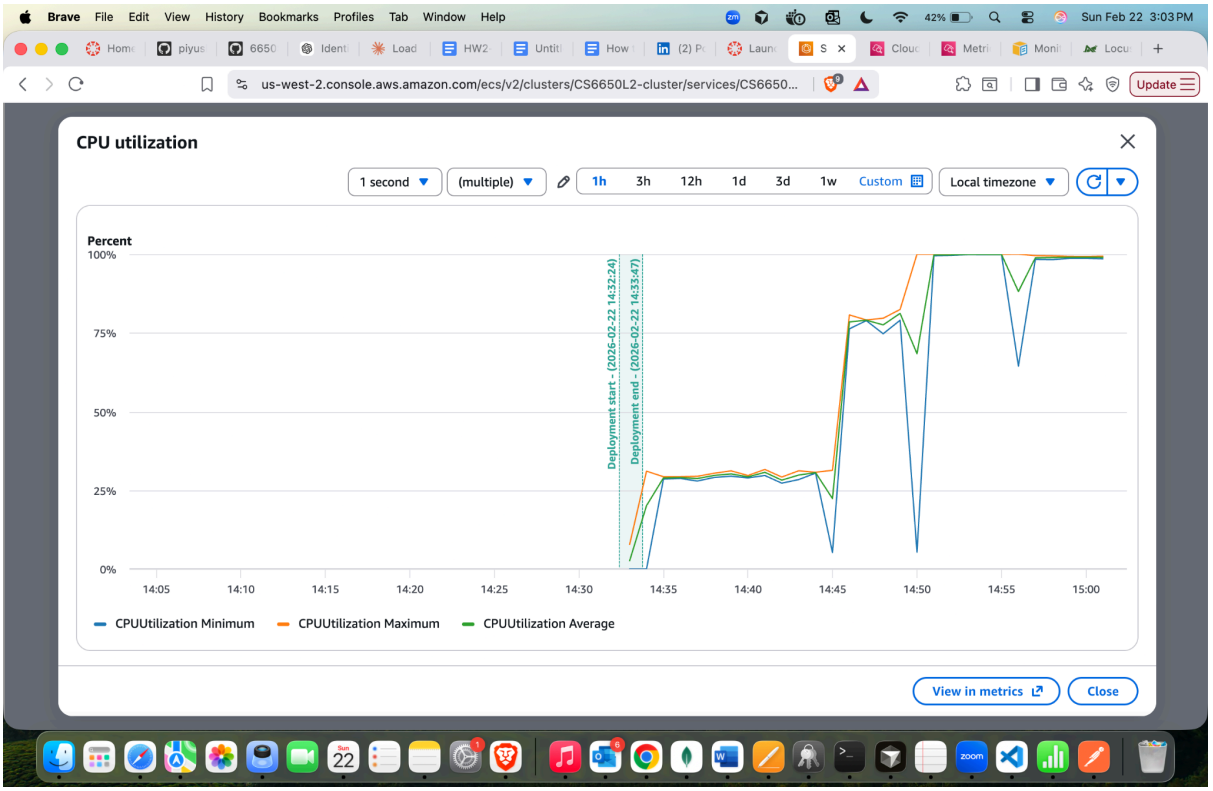
CPU utilization ~5% with 5 users

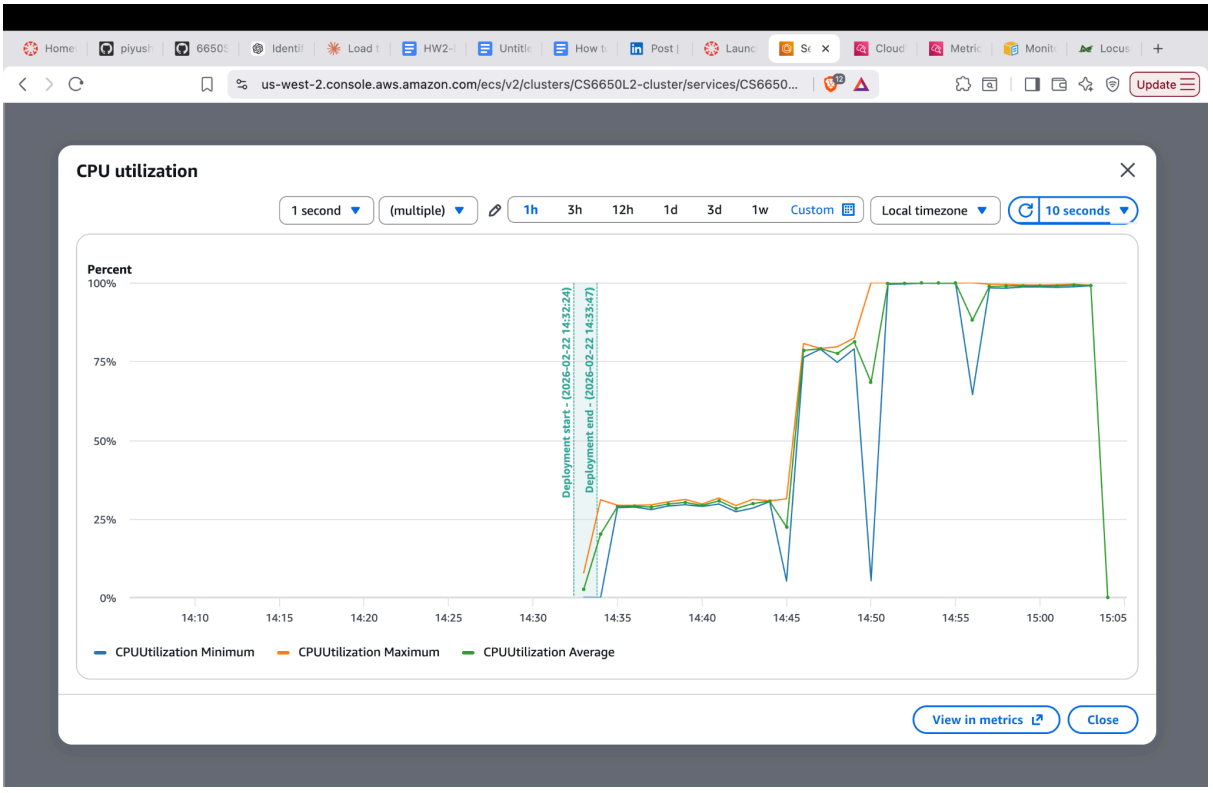


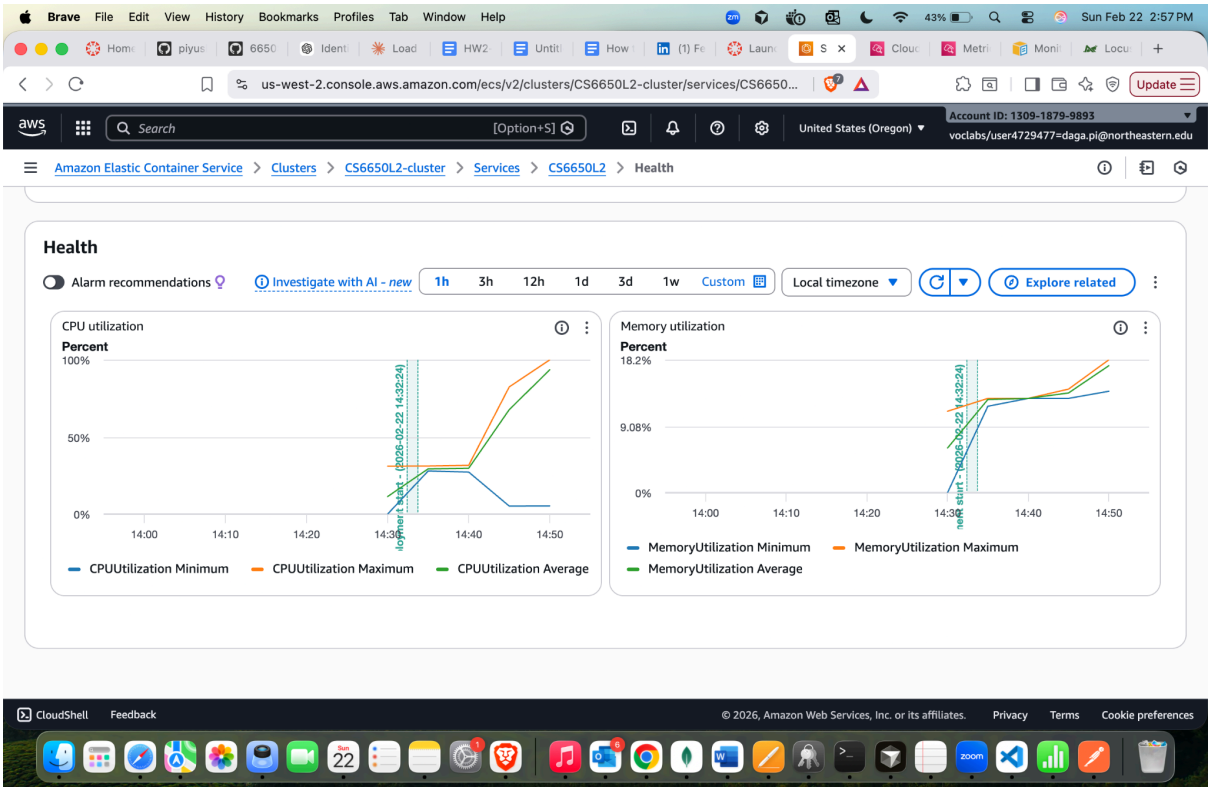
	Users	Avg Response	95th %ile	Max	RPS	CPU	Memory
	5	96ms	100ms	398ms	12.5	~5%	~12%
1	20	96ms	100ms	408ms	49.3	~11%	~13%
	500	126ms	280ms	973ms	1,156	~99.7%	~16%

Changed the wait_time to “between(0, 0)”




CPU utilization
1 second
(multiple)
1h
3h
12h
1d
3d
1w
Custom
Local timezone
Percent
100%
75%
50%
25%
0%
Deployment start - (2026-02-22 14:32:24)
Deployment end - (2026-02-22 14:33:47)
14:05
14:10
14:15
14:20
14:25
14:30
14:35
14:40
14:45
14:50
14:55
15:00
CPUUtilization Minimum
CPUUtilization Maximum
CPUUtilization Average
View in metrics
Close


CPU utilization
1 second
(multiple)
1h
3h
12h
1d
3d
1w
Custom
Local timezone
10 seconds
Percent
100%
75%
50%
25%
0%
Deployment start - (2026-02-22 14:32:24)
Deployment end - (2026-02-22 14:33:47)
14:10
14:15
14:20
14:25
14:30
14:35
14:40
14:45
14:50
14:55
15:00
15:05
CPUUtilization Minimum
CPUUtilization Maximum
CPUUtilization Average
View in metrics
Close



1. What Happened When Load Increased

At 20 users, the service was comfortable fast responses and low CPU. At 100 users, CPU climbed to 80%, signaling strain. At 200 users, the system hit its breaking point CPU reached 100%, average response times gone from 95ms to 123ms. Despite this severe degradation, the service maintained zero failures across all requests.

2. Evidence: Optimization or Scaling?

The evidence clearly points to a **scaling problem, not an optimization problem**:

CPU is the bottleneck, not memory. CloudWatch showed CPU hitting 100% while memory stayed till 18% throughout all tests. Products are loaded once at startup and memory does not grow with traffic. This CPU-bound pattern with stable memory means the system needs more compute power.

The response time increase under load (95ms → 123ms) is caused by requests queuing on a saturated CPU, not by slow code.

when doing fixed-cost computation, the solution is more compute power (scaling), not code optimization.

3. Using CloudWatch Metrics for Scaling Decisions

CloudWatch metrics provided clear signals for scaling decisions:

CPU Utilization is the primary scaling indicator. The progression from 30% (20 users) → 80% (100 users) → 100% (200 users) shows When average CPU consistently exceeds 70-80%, it is time to scale.

Based on these metrics, the recommended action is horizontal scaling — adding more instances behind a load balancer with auto scaling triggered by CPU utilization, which is explored in Part 3.

4. Creative Stress Testing

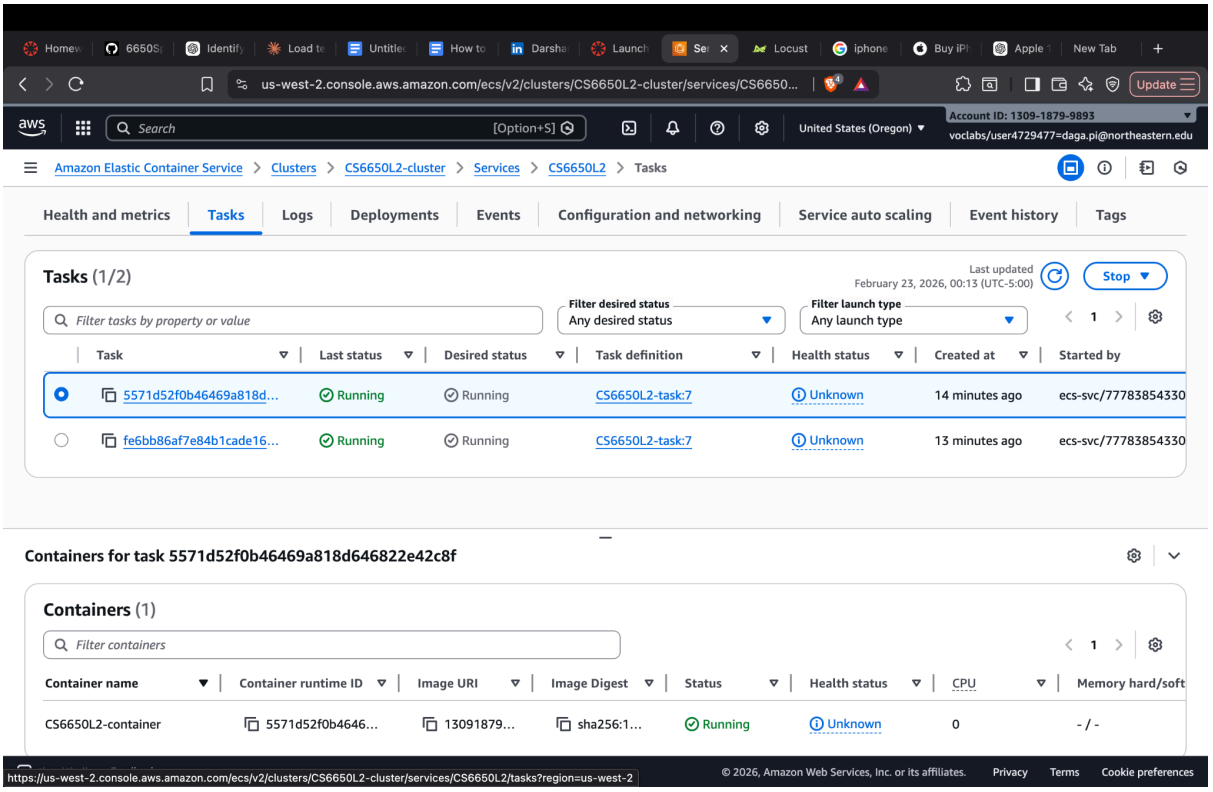
Rather than running the two prescribed tests only, progressive load testing was conducted across multiple runs (20, 100, and 500 users)

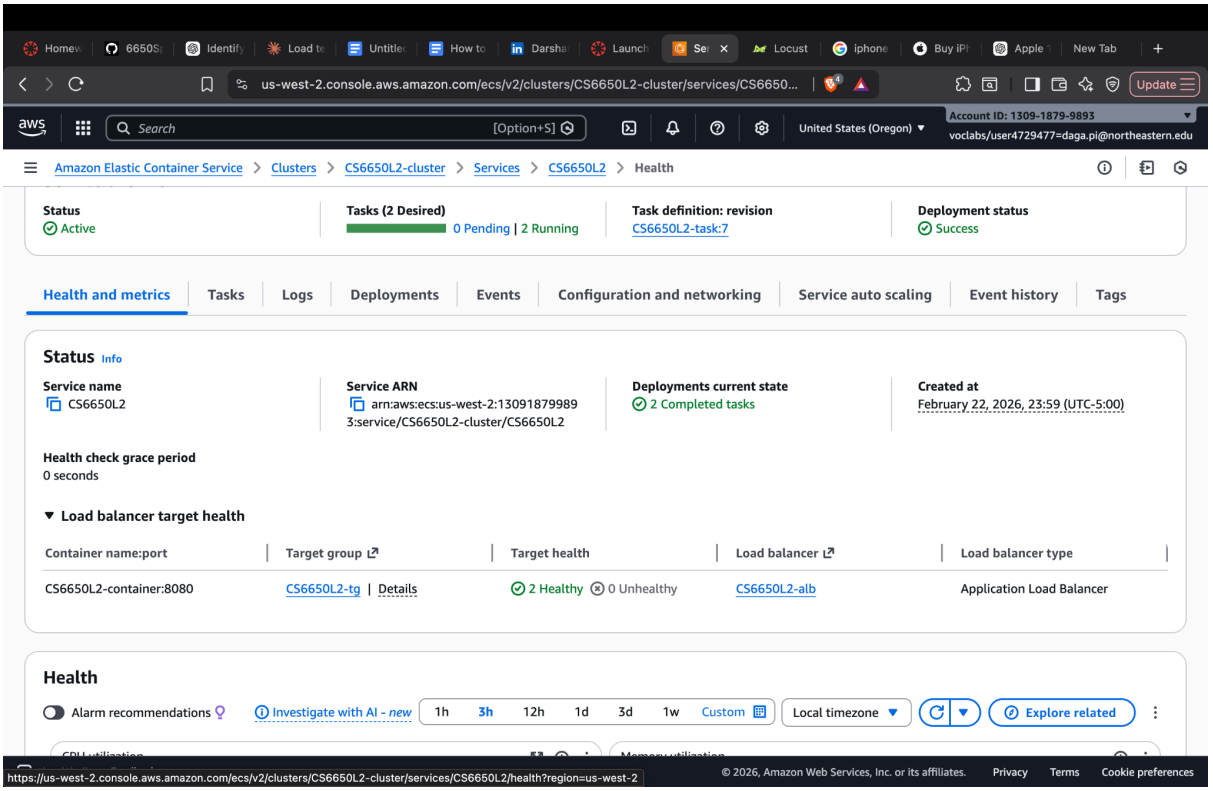
Part 3: Horizontal Scaling with Auto Scaling

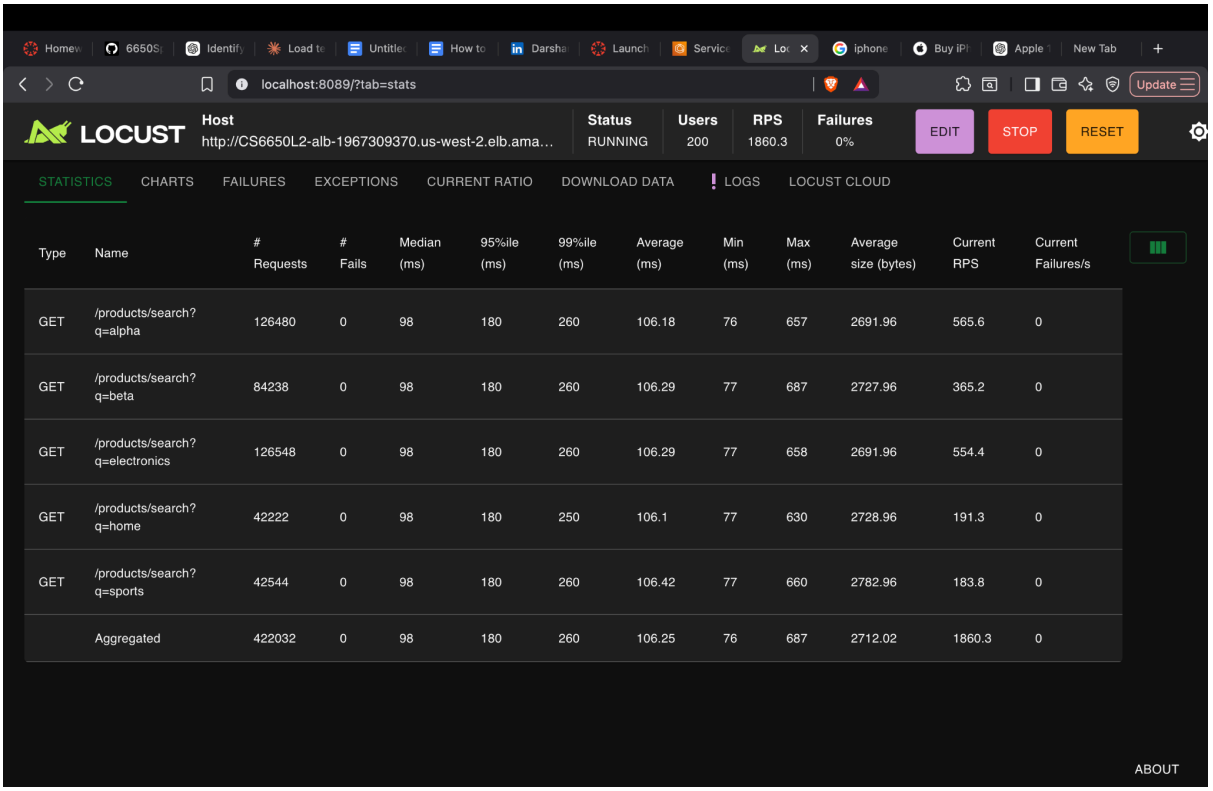
1. How the System Solved the Part II Bottleneck

In Part 2, a single instance (0.25 vCPU) hit 100% CPU at 200 users. The solution was horizontal scaling deploying multiple instances behind an Application Load Balancer with auto scaling.

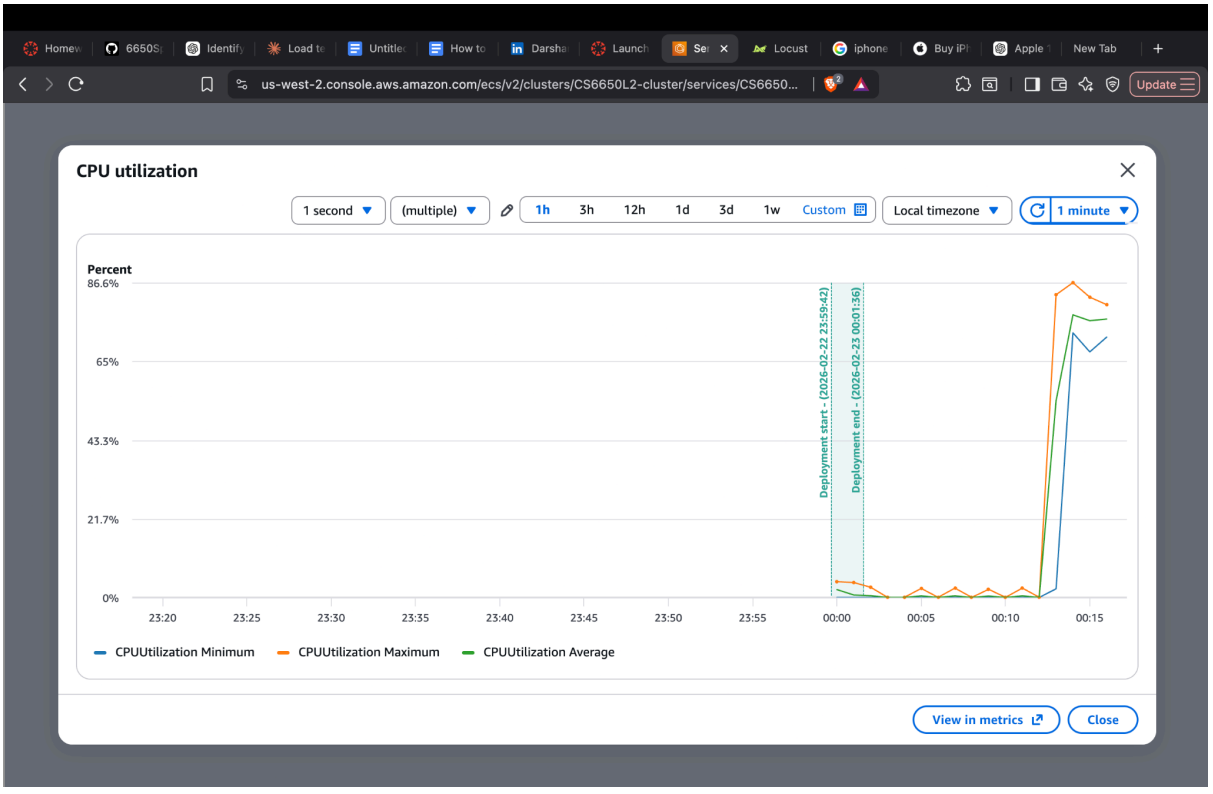
The same service was redeployed with 2 minimum instances, max 4, and a 70% CPU auto scaling target. The results with the same load were dramatically better:


us-west-2.console.aws.amazon.com/ecs/v2/clusters/CS6650L2-cluster/services/CS6650...
Search
[Option+S]
United States (Oregon)
Account ID: 1309-1879-9893
voclabs/user4729477=daga.pi@northeastern.edu
Amazon Elastic Container Service > Clusters > CS6650L2-cluster > Services > CS6650L2 > Tasks
Health and metrics | Tasks | Logs | Deployments | Events | Configuration and networking | Service auto scaling | Event history | Tags
Tasks (1/2)
Last updated February 23, 2026, 00:13 (UTC-5:00)
Stop
Filter tasks by property or value
Filter desired status: Any desired status
Filter launch type: Any launch type
Task | Last status | Desired status | Task definition | Health status | Created at | Started by
5571d52f0b46469a818d... | Running | Running | CS6650L2-task:7 | Unknown | 14 minutes ago | ecs-svc/77783854330
fe6bb86af7e84b1cade16... | Running | Running | CS6650L2-task:7 | Unknown | 13 minutes ago | ecs-svc/77783854330
Containers for task 5571d52f0b46469a818d646822e42c8f
Containers (1)
Filter containers
Container name | Container runtime ID | Image URI | Image Digest | Status | Health status | CPU | Memory hard/soft
CS6650L2-container | 5571d52f0b4646... | 13091879... | sha256:1... | Running | Unknown | 0 | - / -
https://us-west-2.console.aws.amazon.com/ecs/v2/clusters/CS6650L2-cluster/services/CS6650L2/tasks?region=us-west-2
© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

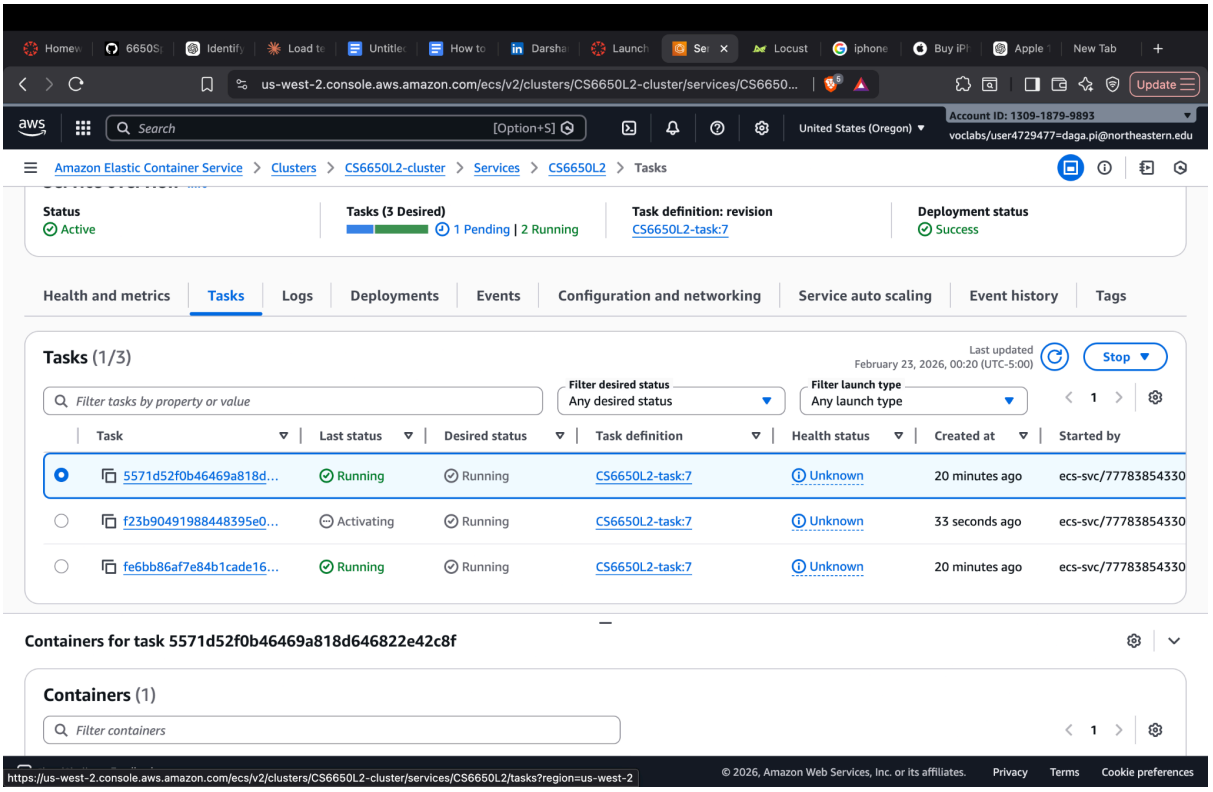

us-west-2.console.aws.amazon.com/ecs/v2/clusters/CS6650L2-cluster/services/CS6650...
Search
[Option+S]
United States (Oregon)
Account ID: 1309-1879-9893
voclabs/user4729477=daga.pi@northeastern.edu
Amazon Elastic Container Service > Clusters > CS6650L2-cluster > Services > CS6650L2 > Health
Status: Active
Tasks (2 Desired): 0 Pending | 2 Running
Task definition: revision: CS6650L2-task:7
Deployment status: Success
Health and metrics | Tasks | Logs | Deployments | Events | Configuration and networking | Service auto scaling | Event history | Tags
Status Info
Service name: CS6650L2
Service ARN: arn:aws:ecs:us-west-2:130918799893:service/CS6650L2-cluster/CS6650L2
Deployments current state: 2 Completed tasks
Created at: February 22, 2026, 23:59 (UTC-5:00)
Health check grace period: 0 seconds
Load balancer target health
Container name:port | Target group | Target health | Load balancer | Load balancer type
CS6650L2-container:8080 | CS6650L2-tg | Details | 2 Healthy 0 Unhealthy | CS6650L2-alb | Application Load Balancer
Health
Alarm recommendations
Investigate with AI - new
1h 3h 12h 1d 3d 1w Custom
Local timezone
Explore related
https://us-west-2.console.aws.amazon.com/ecs/v2/clusters/CS6650L2-cluster/services/CS6650L2/health?region=us-west-2
© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

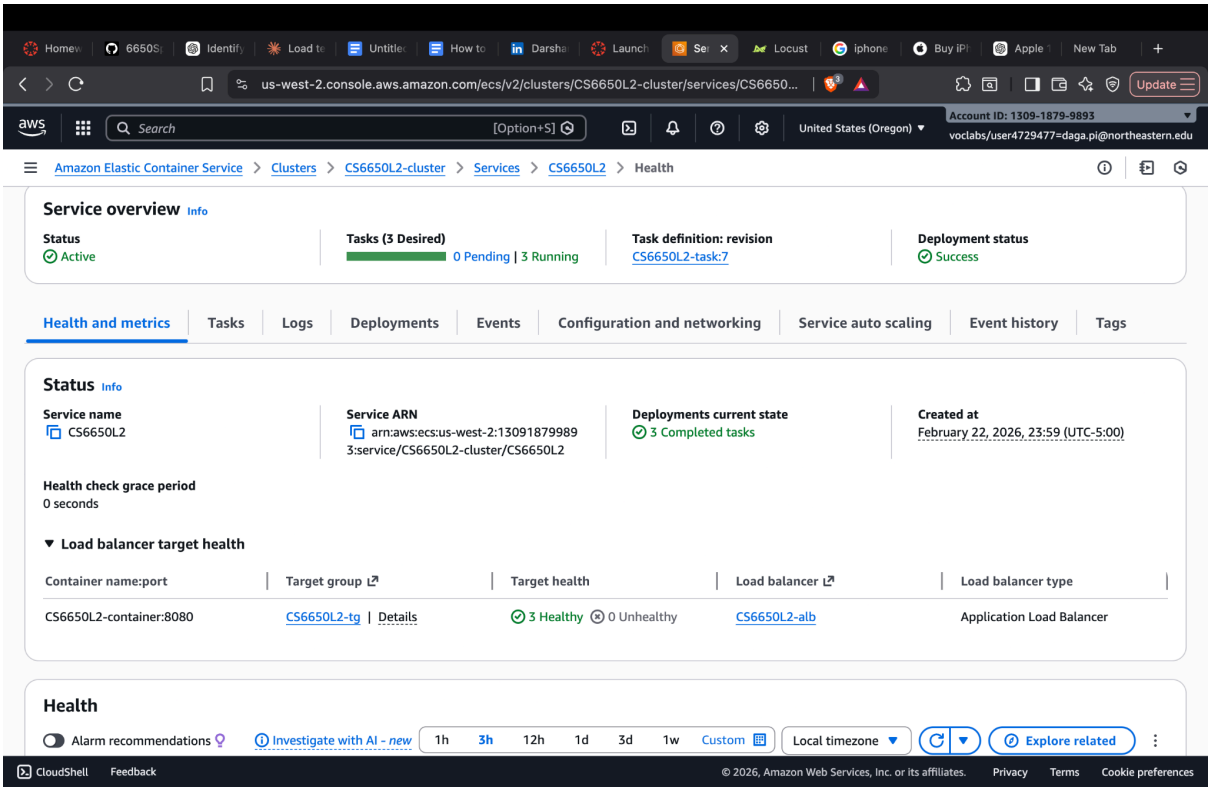


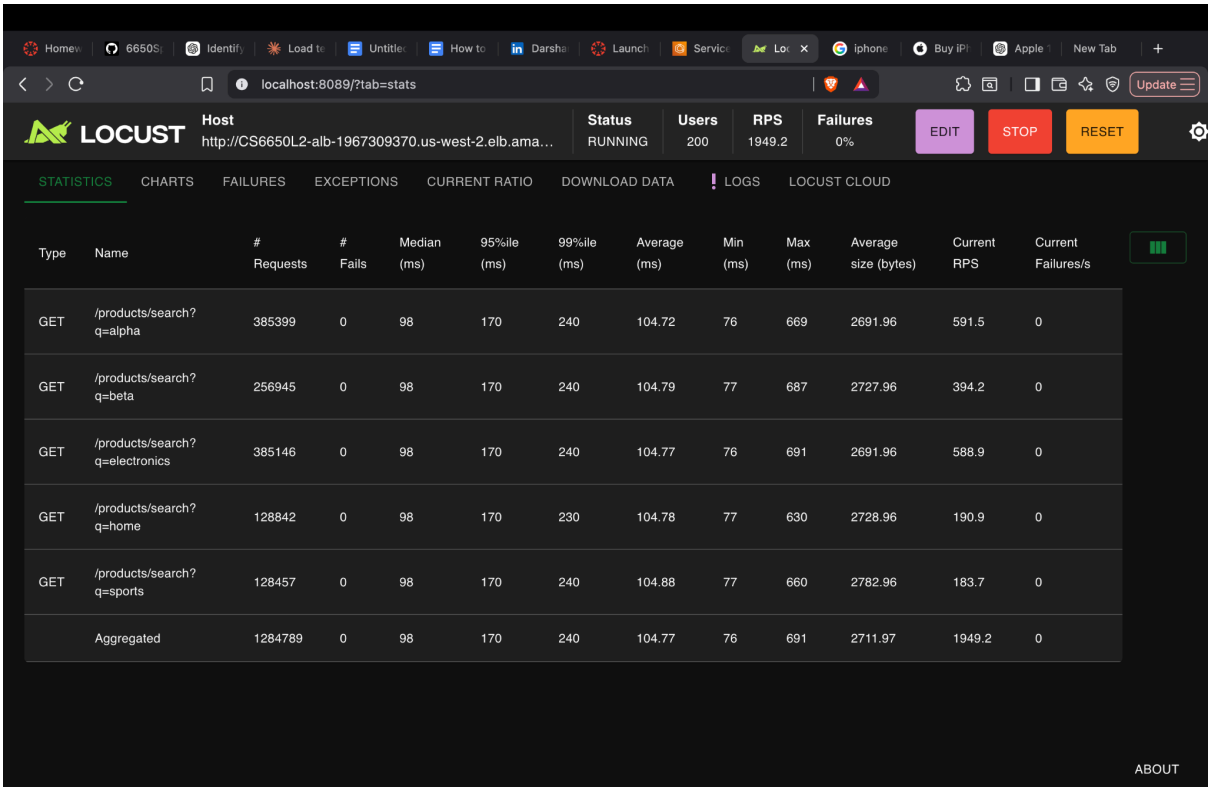
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/search?q=alpha	126480	0	98	180	260	106.18	76	657	2691.96	565.6	0
GET	/products/search?q=beta	84238	0	98	180	260	106.29	77	687	2727.96	365.2	0
GET	/products/search?q=electronics	126548	0	98	180	260	106.29	77	658	2691.96	554.4	0
GET	/products/search?q=home	42222	0	98	180	250	106.1	77	630	2728.96	191.3	0
GET	/products/search?q=sports	42544	0	98	180	260	106.42	77	660	2782.96	183.8	0
	Aggregated	422032	0	98	180	260	106.25	76	687	2712.02	1860.3	0



Autoscaling creates the 3rd instance


us-west-2.console.aws.amazon.com/ecs/v2/clusters/CS6650L2-cluster/services/CS6650...
Search
[Option+S]
United States (Oregon)
Account ID: 1309-1879-9893
voclabs/user4729477=daga.pi@northeastern.edu
Amazon Elastic Container Service > Clusters > CS6650L2-cluster > Services > CS6650L2 > Tasks
Status
Active
Tasks (3 Desired)
1 Pending | 2 Running
Task definition: revision
CS6650L2-task:7
Deployment status
Success
Health and metrics
Tasks
Logs
Deployments
Events
Configuration and networking
Service auto scaling
Event history
Tags
Tasks (1/3)
Last updated
February 23, 2026, 00:20 (UTC-5:00)
Stop
Filter tasks by property or value
Filter desired status
Any desired status
Filter launch type
Any launch type
Task
Last status
Desired status
Task definition
Health status
Created at
Started by
5571d52f0b46469a818d...
Running
Running
CS6650L2-task:7
Unknown
20 minutes ago
ecs-svc/77783854330
f23b90491988448395e0...
Activating
Running
CS6650L2-task:7
Unknown
33 seconds ago
ecs-svc/77783854330
fe6bb86af7e84b1cade16...
Running
Running
CS6650L2-task:7
Unknown
20 minutes ago
ecs-svc/77783854330
Containers for task 5571d52f0b46469a818d646822e42c8f
Containers (1)
Filter containers
https://us-west-2.console.aws.amazon.com/ecs/v2/clusters/CS6650L2-cluster/services/CS6650L2/tasks?region=us-west-2
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


us-west-2.console.aws.amazon.com/ecs/v2/clusters/CS6650L2-cluster/services/CS6650...
Search
[Option+S]
United States (Oregon)
Account ID: 1309-1879-9893
voclabs/user4729477=daga.pi@northeastern.edu
Amazon Elastic Container Service > Clusters > CS6650L2-cluster > Services > CS6650L2 > Health
Service overview Info
Status
Active
Tasks (3 Desired)
0 Pending | 3 Running
Task definition: revision
CS6650L2-task:7
Deployment status
Success
Health and metrics
Tasks
Logs
Deployments
Events
Configuration and networking
Service auto scaling
Event history
Tags
Status Info
Service name
CS6650L2
Service ARN
arn:aws:ecs:us-west-2:130918799893:service/CS6650L2-cluster/CS6650L2
Deployments current state
3 Completed tasks
Created at
February 22, 2026, 23:59 (UTC-5:00)
Health check grace period
0 seconds
Load balancer target health
Container name:port
Target group
Target health
Load balancer
Load balancer type
CS6650L2-container:8080
CS6650L2-tg | Details
3 Healthy
0 Unhealthy
CS6650L2-alb
Application Load Balancer
Health
Alarm recommendations
Investigate with AI - new
1h
3h
12h
1d
3d
1w
Custom
Local timezone
Explore related
CloudShell
Feedback
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences



Response time decrease from 106 to 104

2. Role of Each Component

Application Load Balancer (ALB): Acts as the single entry point for all traffic. Clients use one DNS name regardless of how many instances are running.

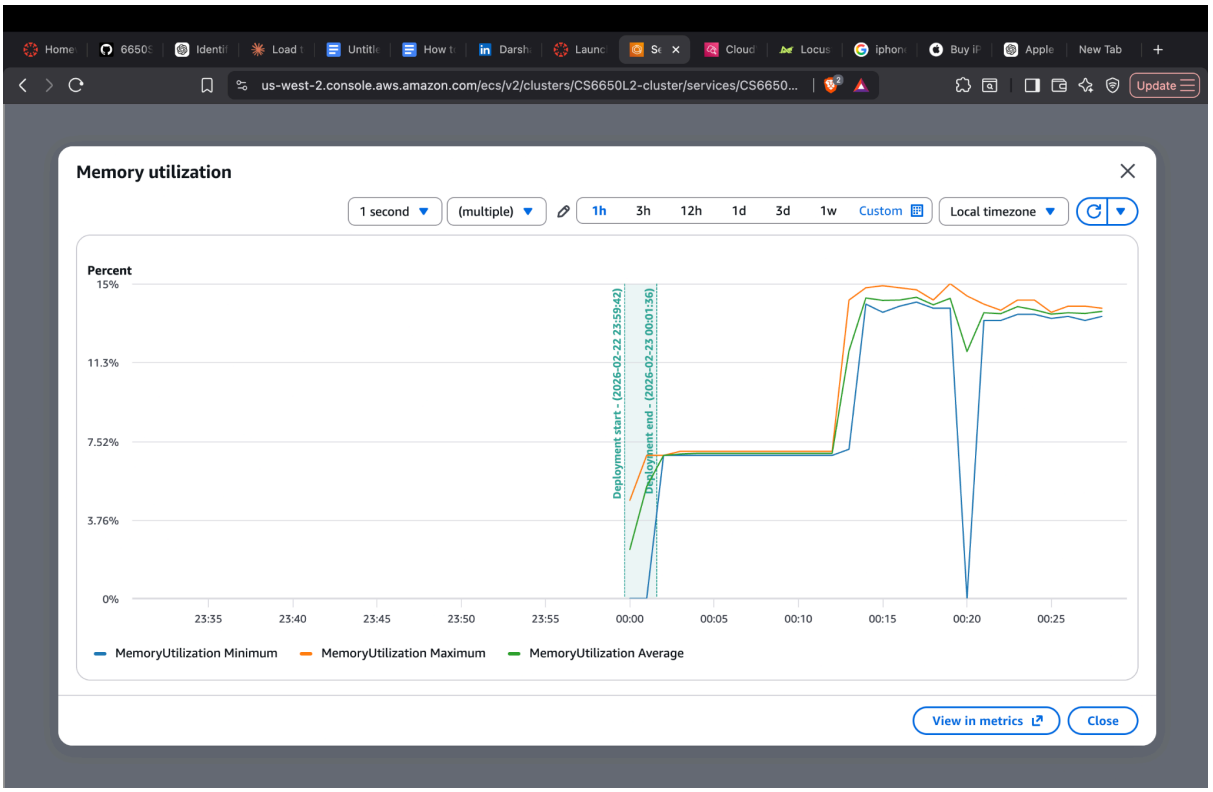
Example;
curl"http://CS6650L2-alb-1967309370.us-west-2.elb.amazonaws.com/products/search?q=alpha"

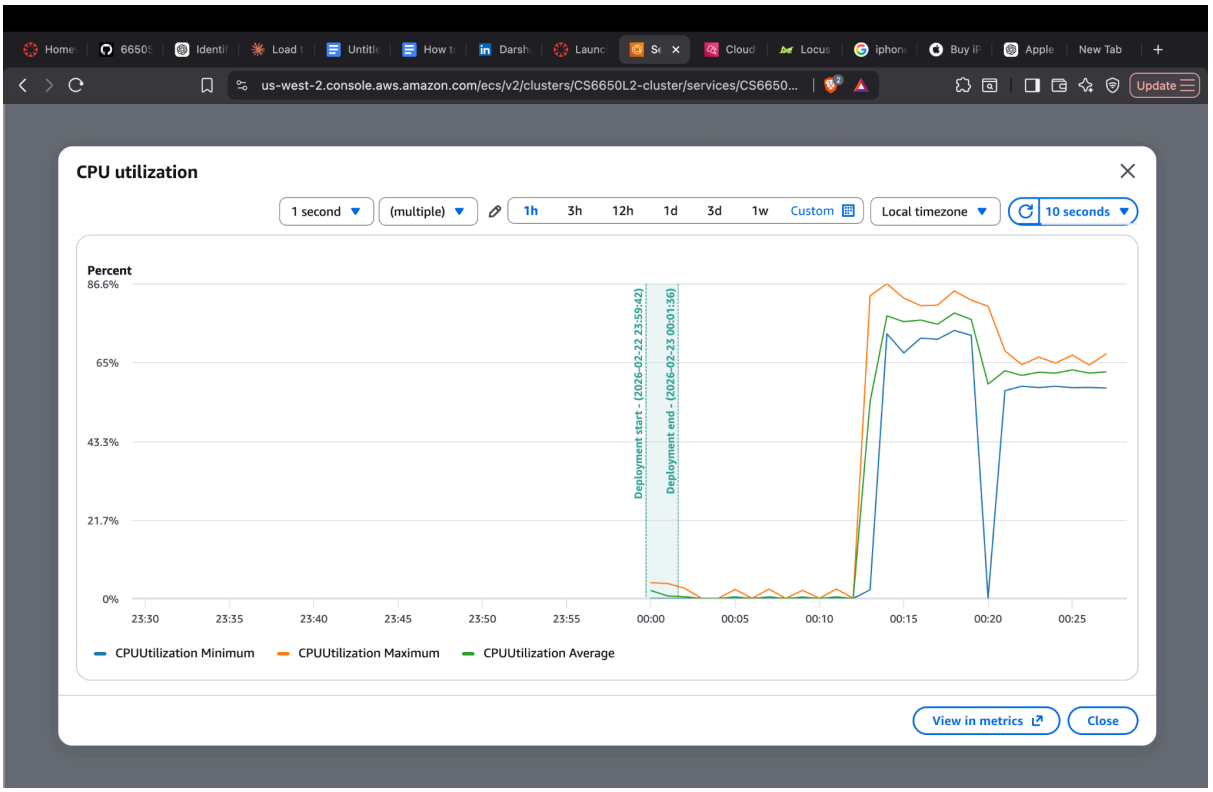
The ALB distributes incoming requests across all healthy instances using round-robin routing. This means adding or removing instances requires any changes on the client side.

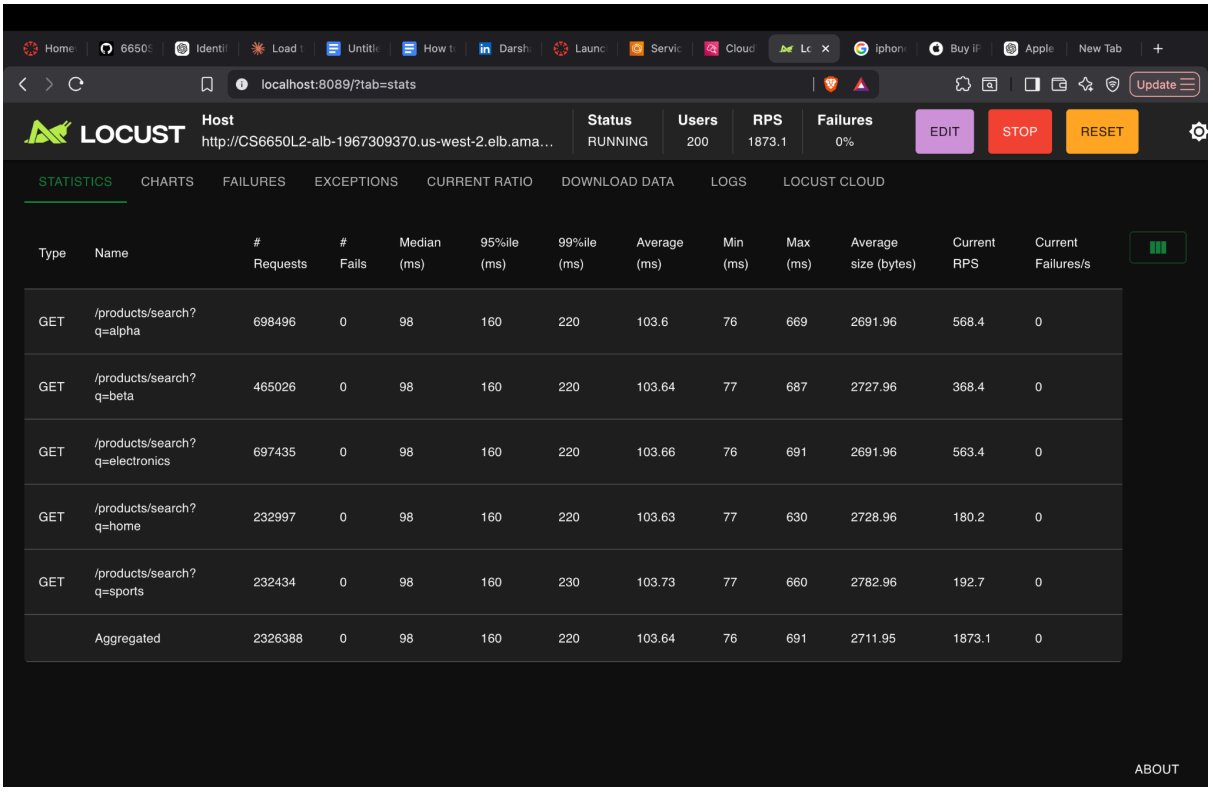
Target Group: Monitors the health of each instance by sending requests to the /health endpoint every 30 seconds. If an instance fails 3 consecutive health checks, it is automatically removed from rotation. When a new instance passes 2 consecutive checks, it is added. This ensures traffic only goes to healthy, responsive instances.

Auto Scaling: Watches the average CPU utilization across all instances. When CPU exceeds the target (70%), it increases the desired task count. When CPU drops below, it decreases the count after a cooldown period.

CPU and memory utilization

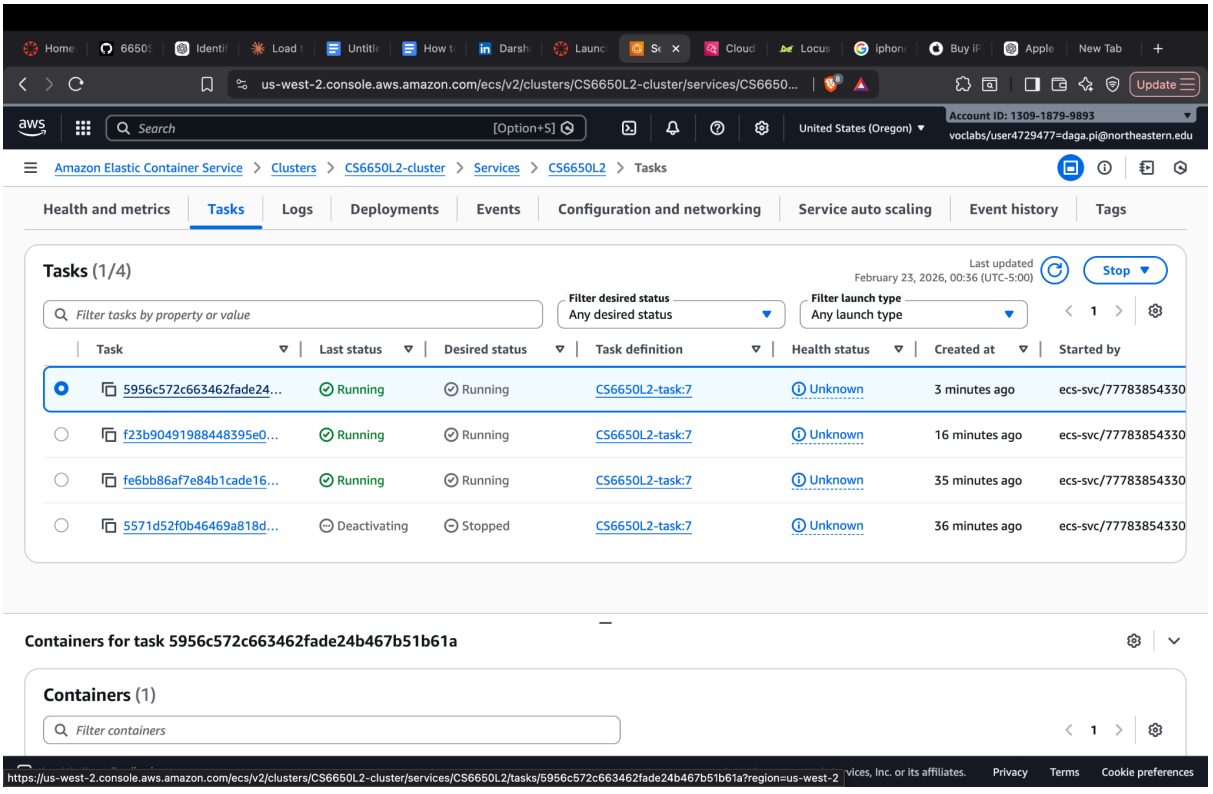






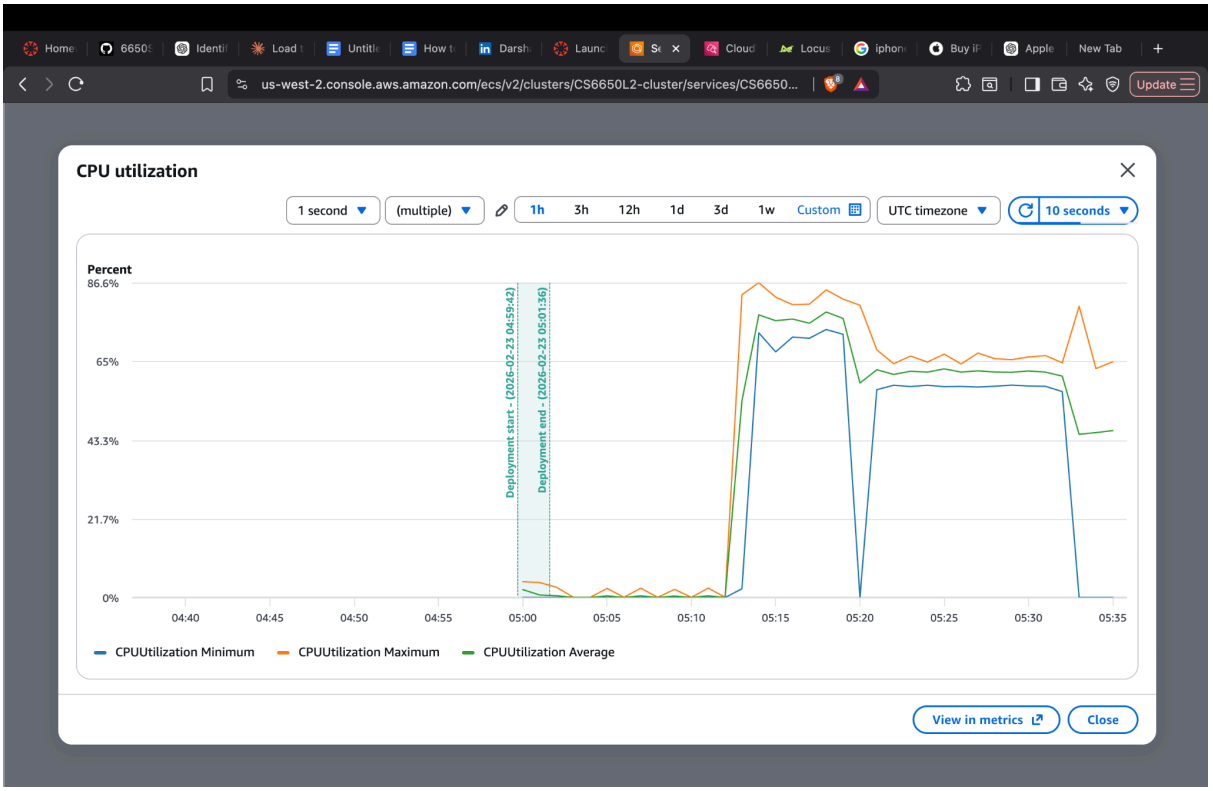
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/search?q=alpha	698496	0	98	160	220	103.6	76	669	2691.96	568.4	0
GET	/products/search?q=beta	465026	0	98	160	220	103.64	77	687	2727.96	368.4	0
GET	/products/search?q=electronics	697435	0	98	160	220	103.66	76	691	2691.96	563.4	0
GET	/products/search?q=home	232997	0	98	160	220	103.63	77	630	2728.96	180.2	0
GET	/products/search?q=sports	232434	0	98	160	230	103.73	77	660	2782.96	192.7	0
	Aggregated	2326388	0	98	160	220	103.64	76	691	2711.95	1873.1	0

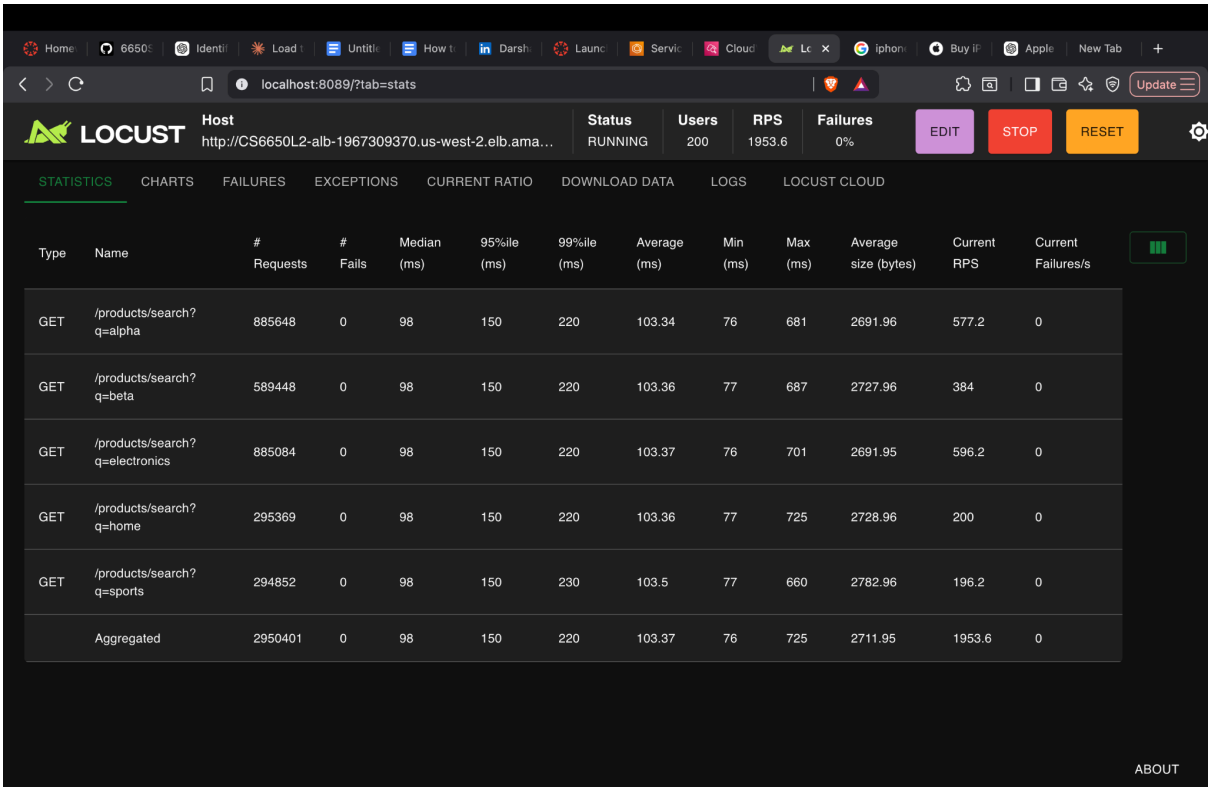
Deactivated the instance and it re created th



Task	Last status	Desired status	Task definition	Health status	Created at	Started by
5956c572c663462fade24...	Running	Running	CS6650L2-task:7	Unknown	3 minutes ago	ecs-svc/77783854330
f23b90491988448395e0...	Running	Running	CS6650L2-task:7	Unknown	16 minutes ago	ecs-svc/77783854330
fe6bb86af7e84b1cade16...	Running	Running	CS6650L2-task:7	Unknown	35 minutes ago	ecs-svc/77783854330
5571d52f0b46469a818d...	Deactivating	Stopped	CS6650L2-task:7	Unknown	36 minutes ago	ecs-svc/77783854330

e instance again in some time

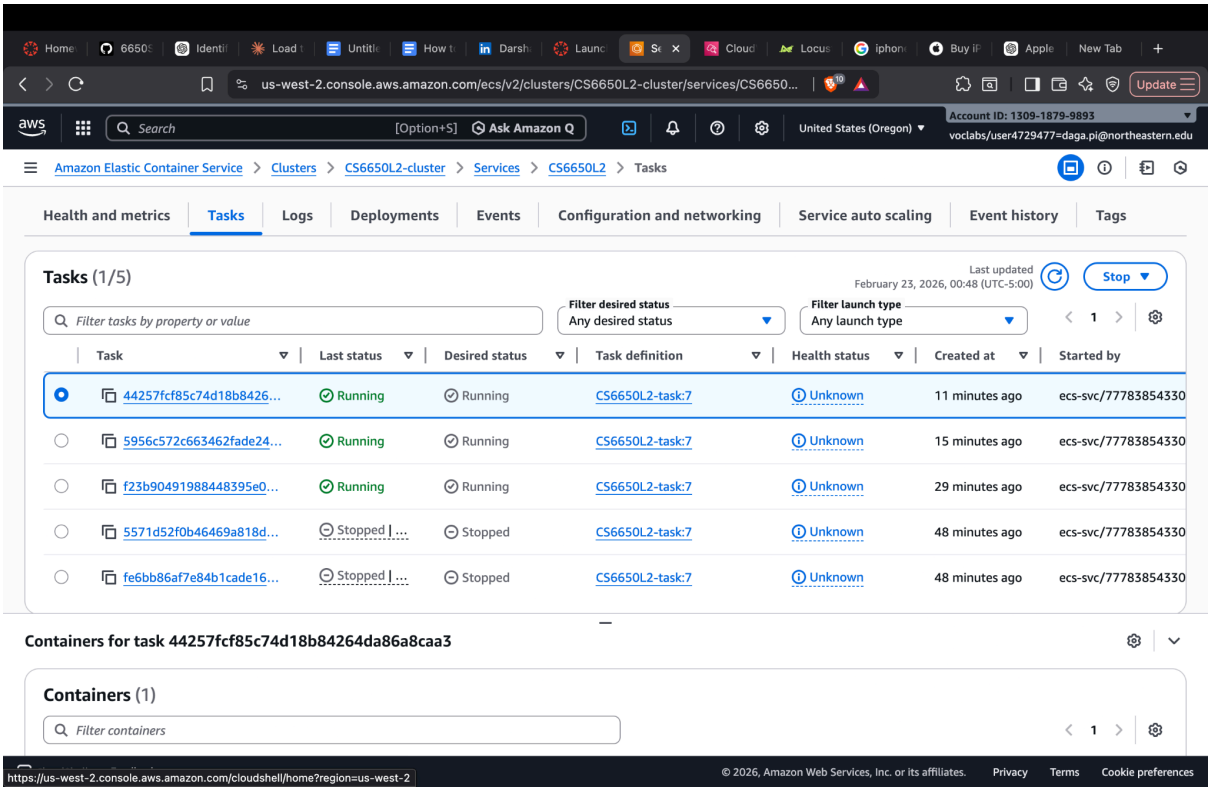


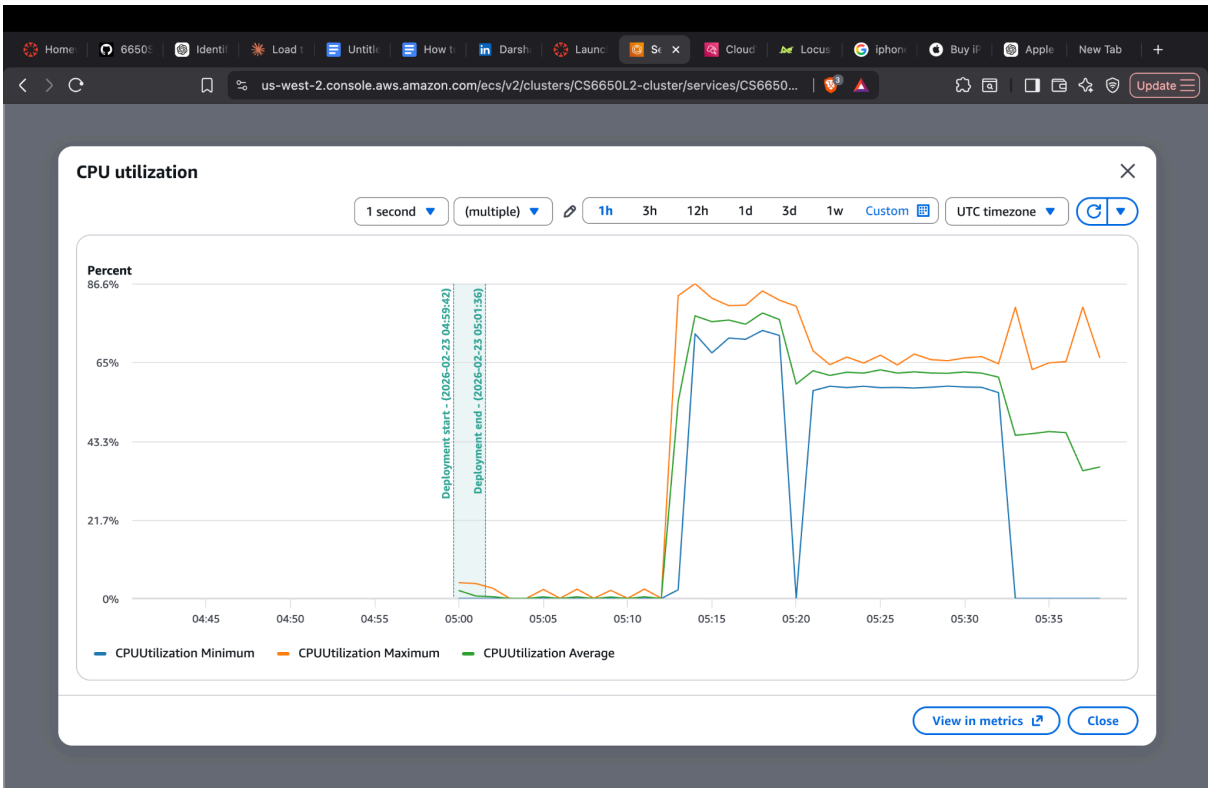


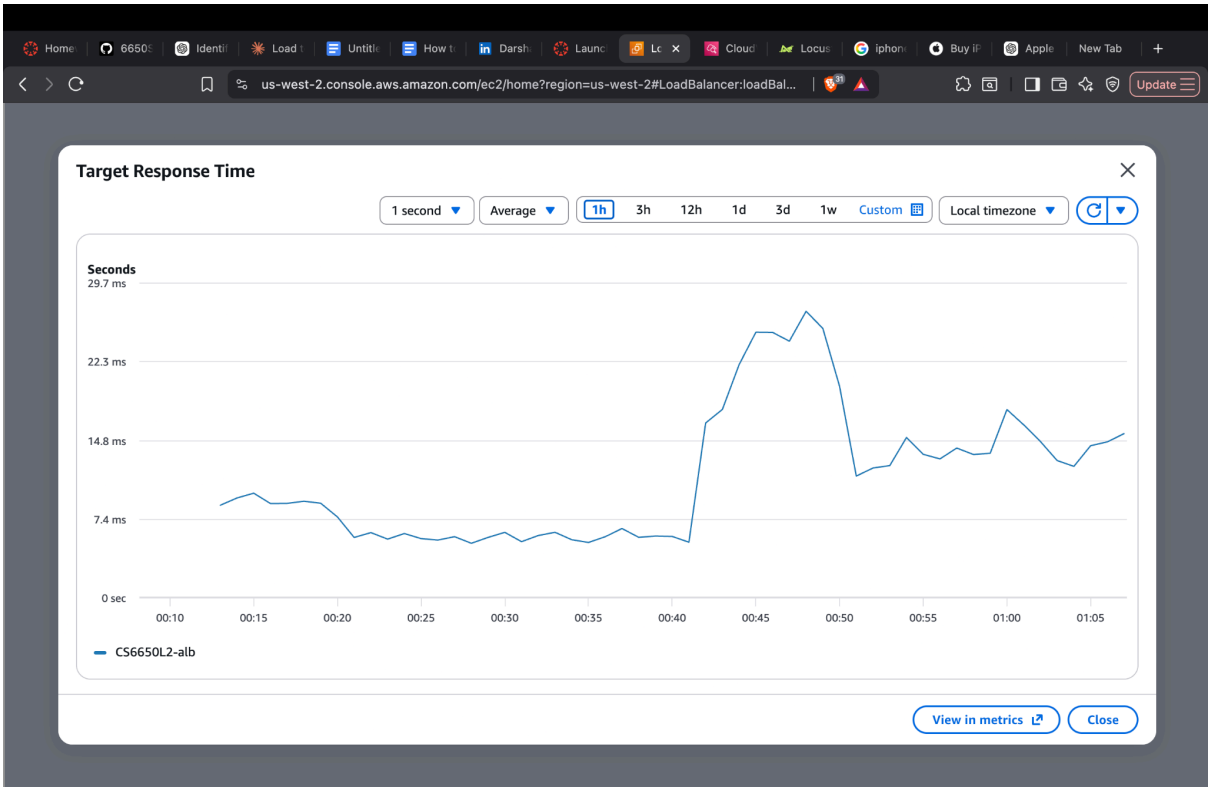
Till now 3 tasks created and the response time is also reduced so nicely balancing the load

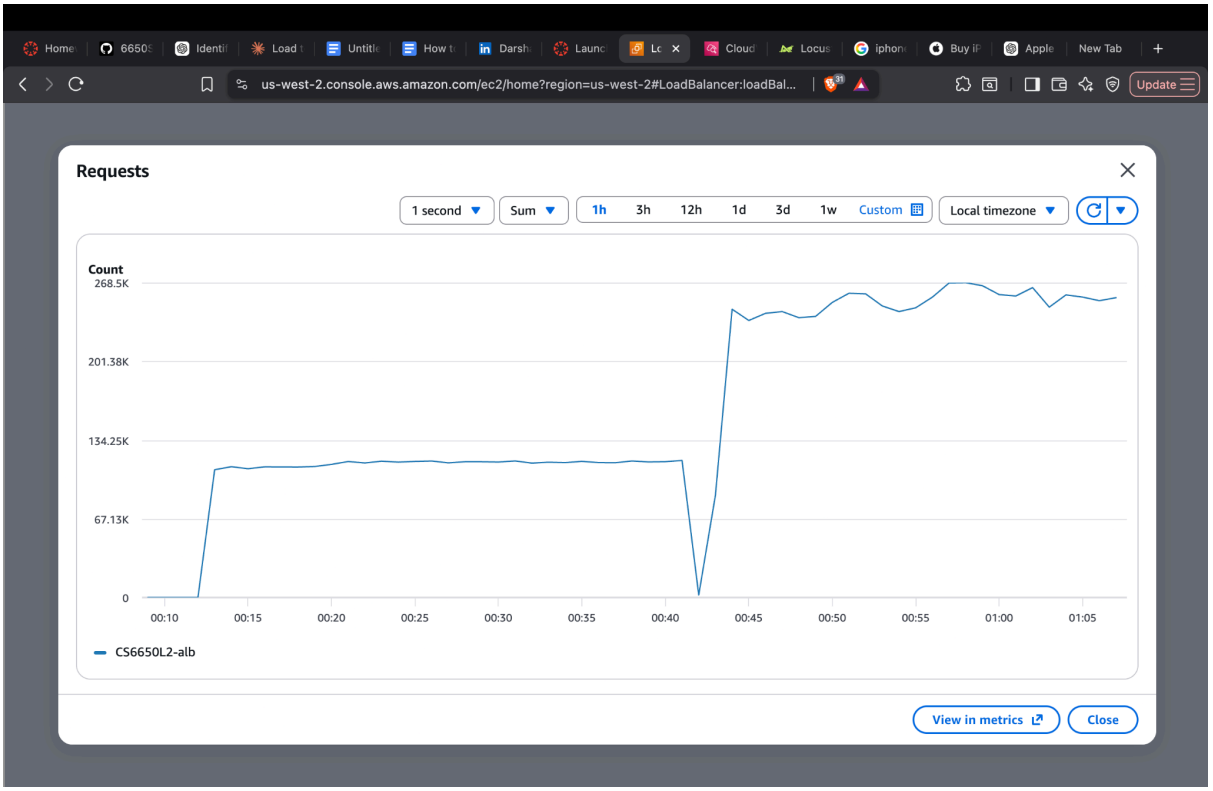
Resilience Testing

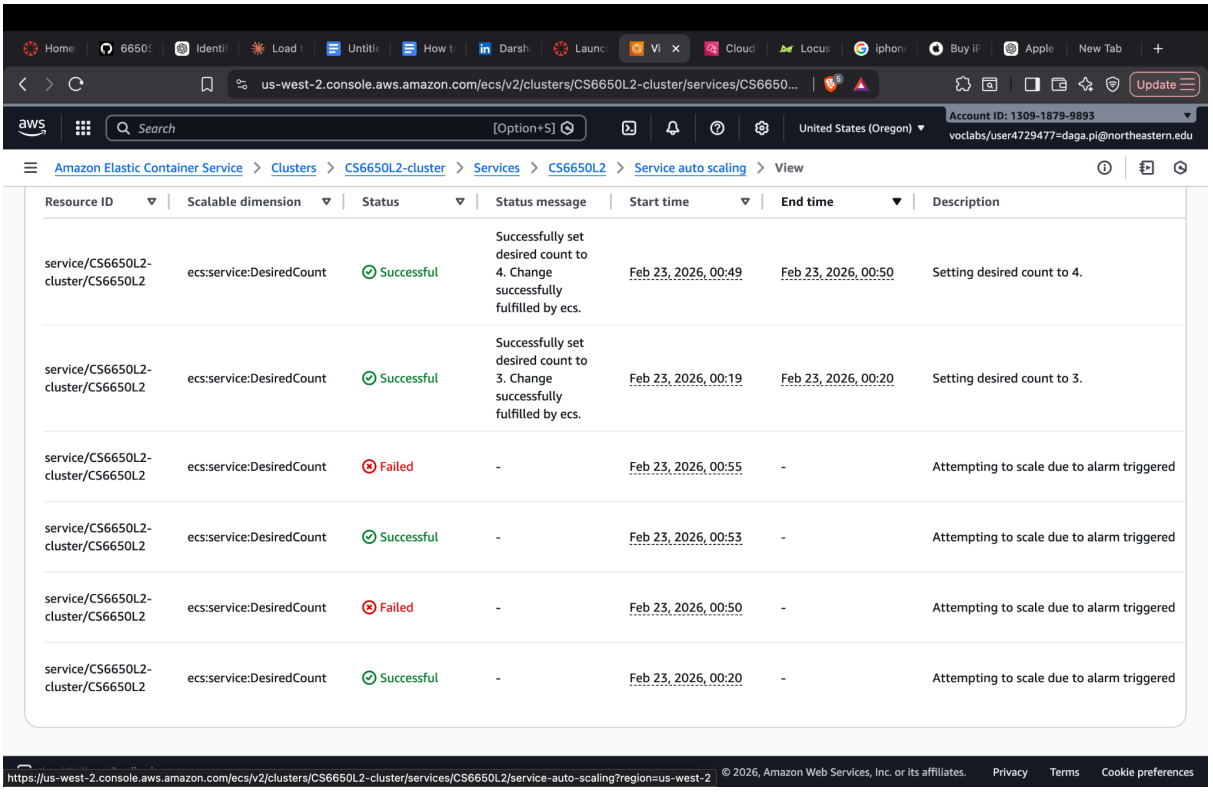
Deleted another task and the LB created another






Target Response Time
1 second
Average
1h
3h
12h
1d
3d
1w
Custom
Local timezone
Seconds
29.7 ms
22.3 ms
14.8 ms
7.4 ms
0 sec
00:10
00:15
00:20
00:25
00:30
00:35
00:40
00:45
00:50
00:55
01:00
01:05
CS6650L2-alb
View in metrics
Close


Requests
1 second
Sum
1h
3h
12h
1d
3d
1w
Custom
Local timezone
Count
268.5K
201.38K
134.25K
67.13K
0
00:10
00:15
00:20
00:25
00:30
00:35
00:40
00:45
00:50
00:55
01:00
01:05
CS6650L2-alb
View in metrics
Close



In the above image even though 2 tasks were stopped the LB created the new ones without completely collapsing the service

3. Trade-offs: Horizontal vs Vertical Scaling

Aspect	Vertical Scaling (bigger instance)	Horizontal Scaling (more instances)
Implementation	increase CPU/memory units	Complex requires ALB, target groups, scaling policies
Fault tolerance	Single point of failure if it dies, service is down	Survives instance failures ALB routes around dead instances
Cost efficiency	Pay for peak capacity 24/7	Scale down during low traffic, pay only for what you use
Scaling limit	Hardware ceiling (max vCPU available)	Nearly unlimited keep adding instances
Downtime	Requires restart to resize	Zero downtime new instances added while old ones serve
Complexity	Low just change a number	Higher need load balancer, health checks, scaling policies

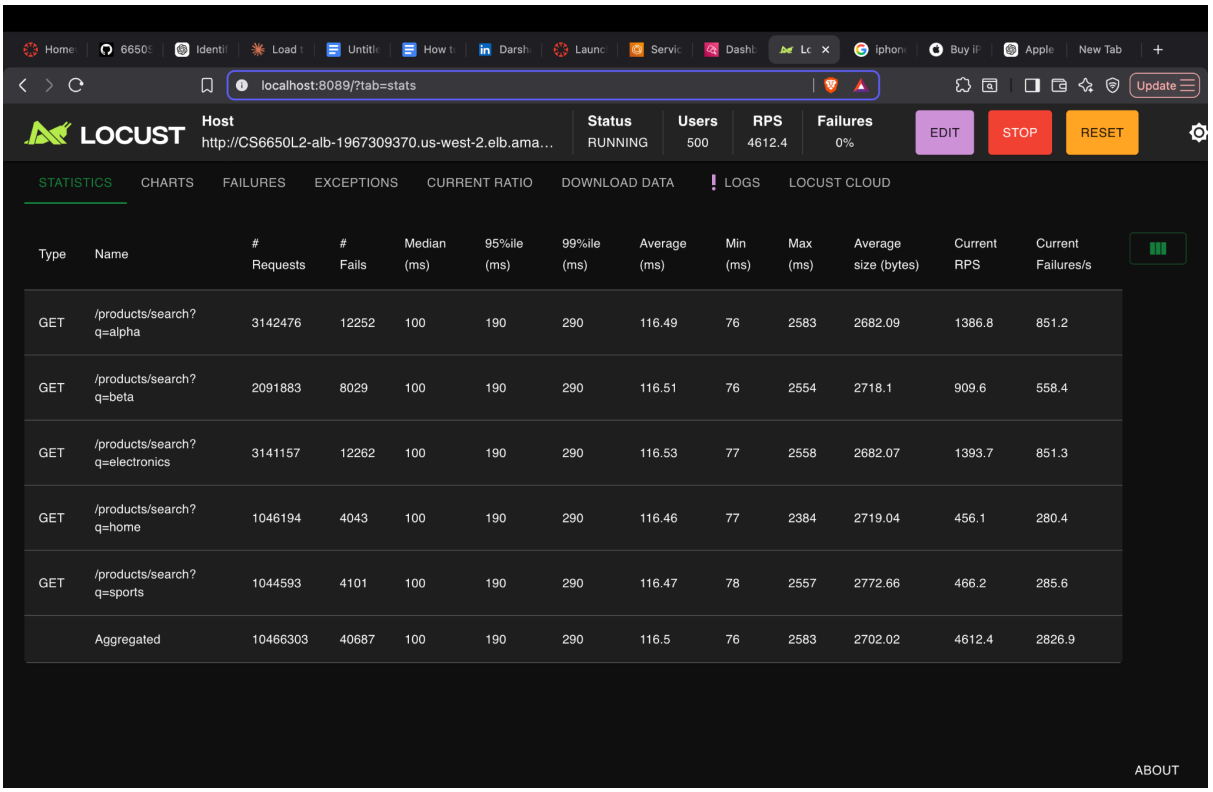
4. Predicting Scaling Behavior for Different Load Patterns

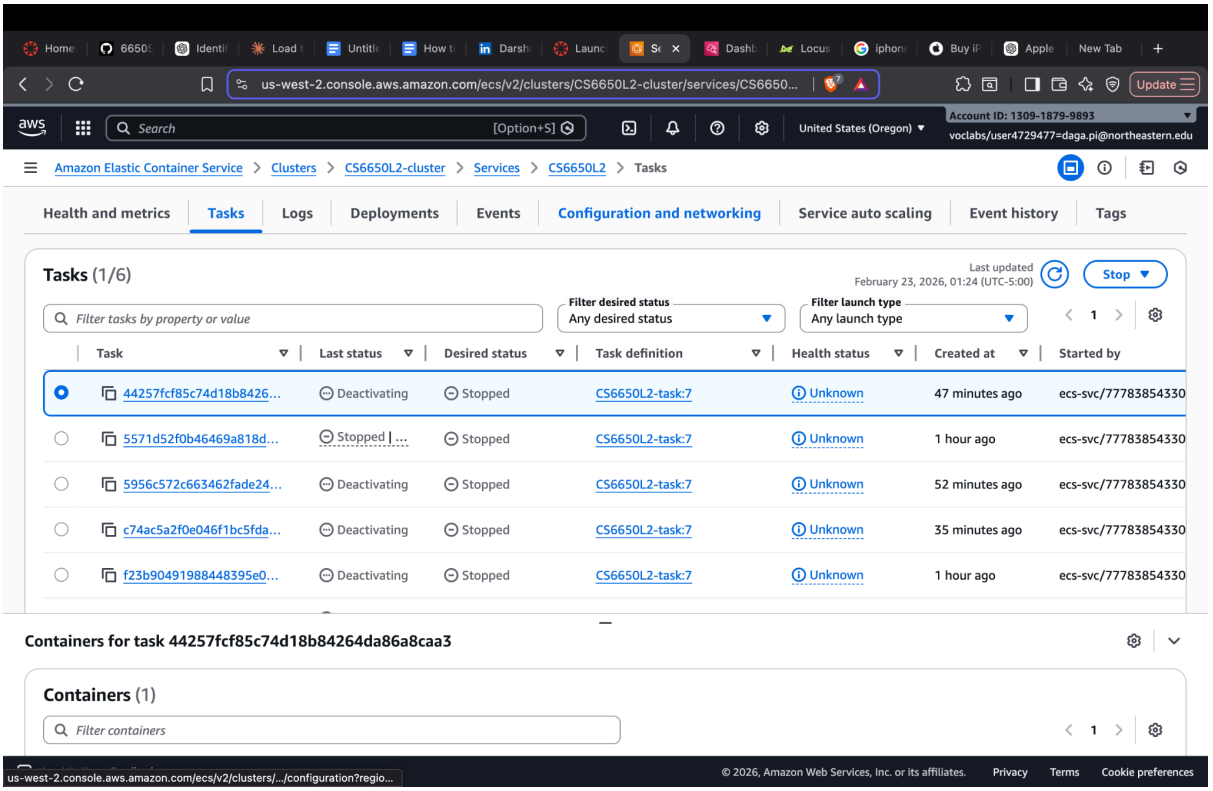
Experimental evidence with different scaling policies:

50% CPU target: With a lower threshold, auto scaling triggered more aggressively. Under 500-user load, the system scaled to 4 tasks and CPU stabilized at lower levels compared to the 70% target. This results in more instances running (higher cost) but better response times.

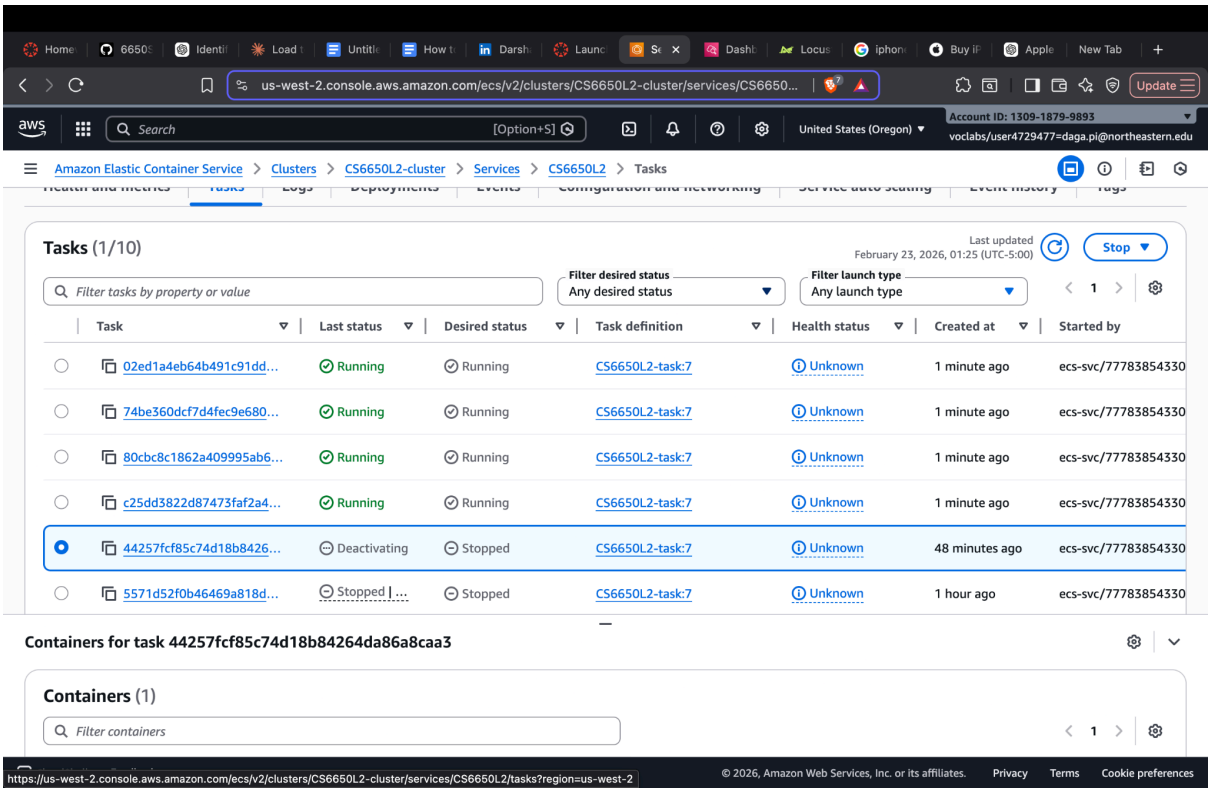
60-second cooldown vs 300-second cooldown: With shorter cooldowns, the system reacted much faster to load changes. Tasks were added within minutes instead of waiting 5+ minutes between scaling events.

All tasks stopped temporarily down but the LB created the new tasks soon

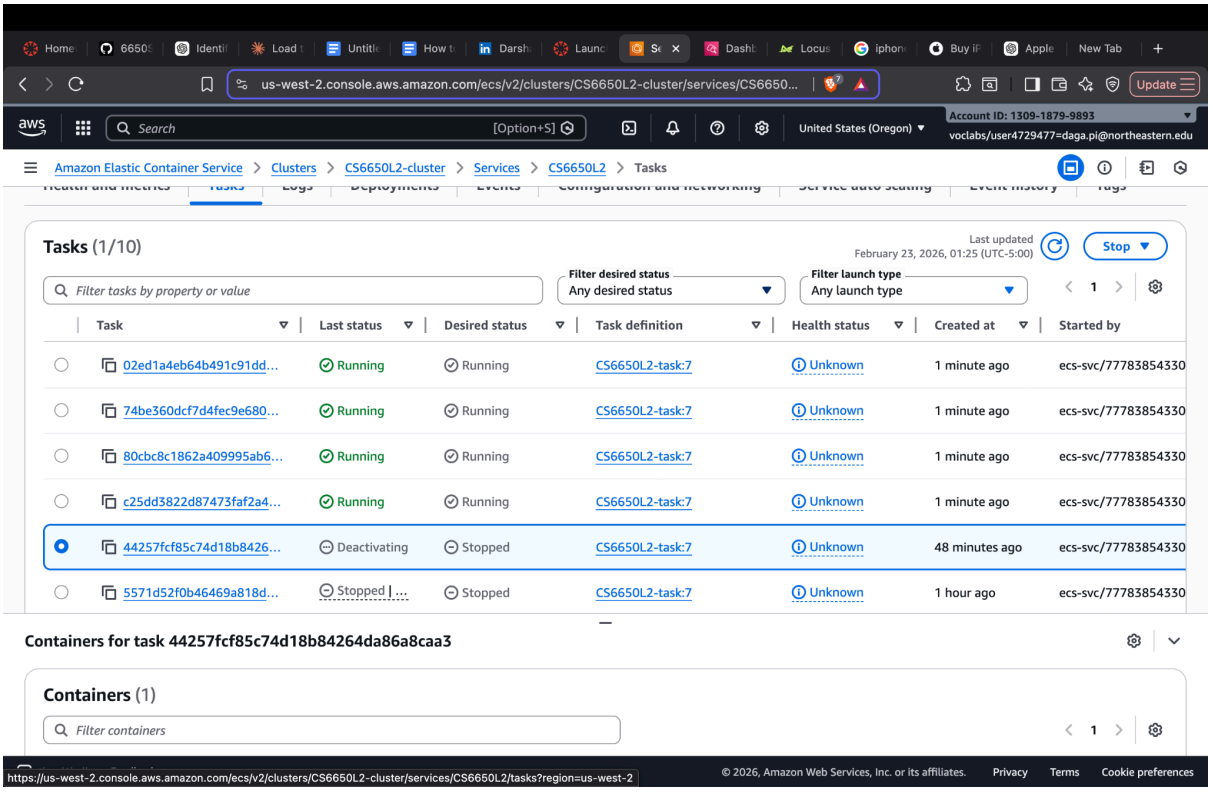


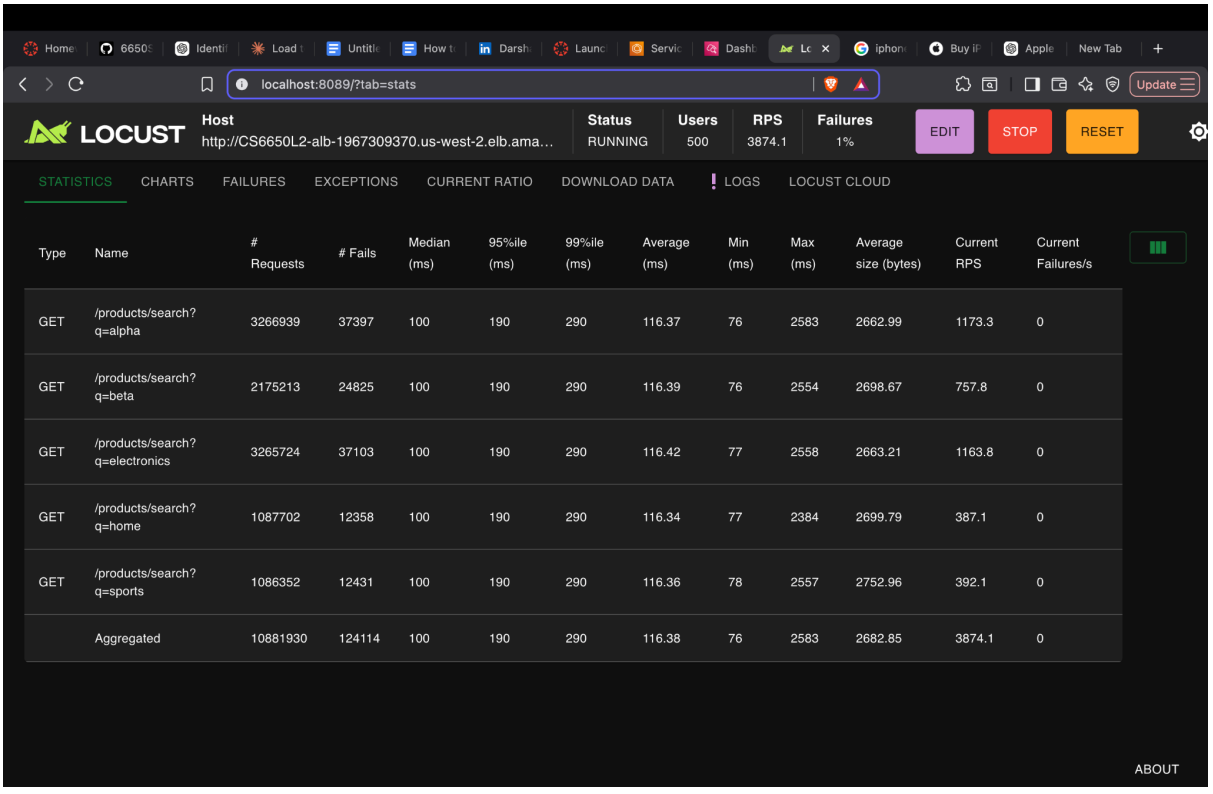


Created the new tasks again

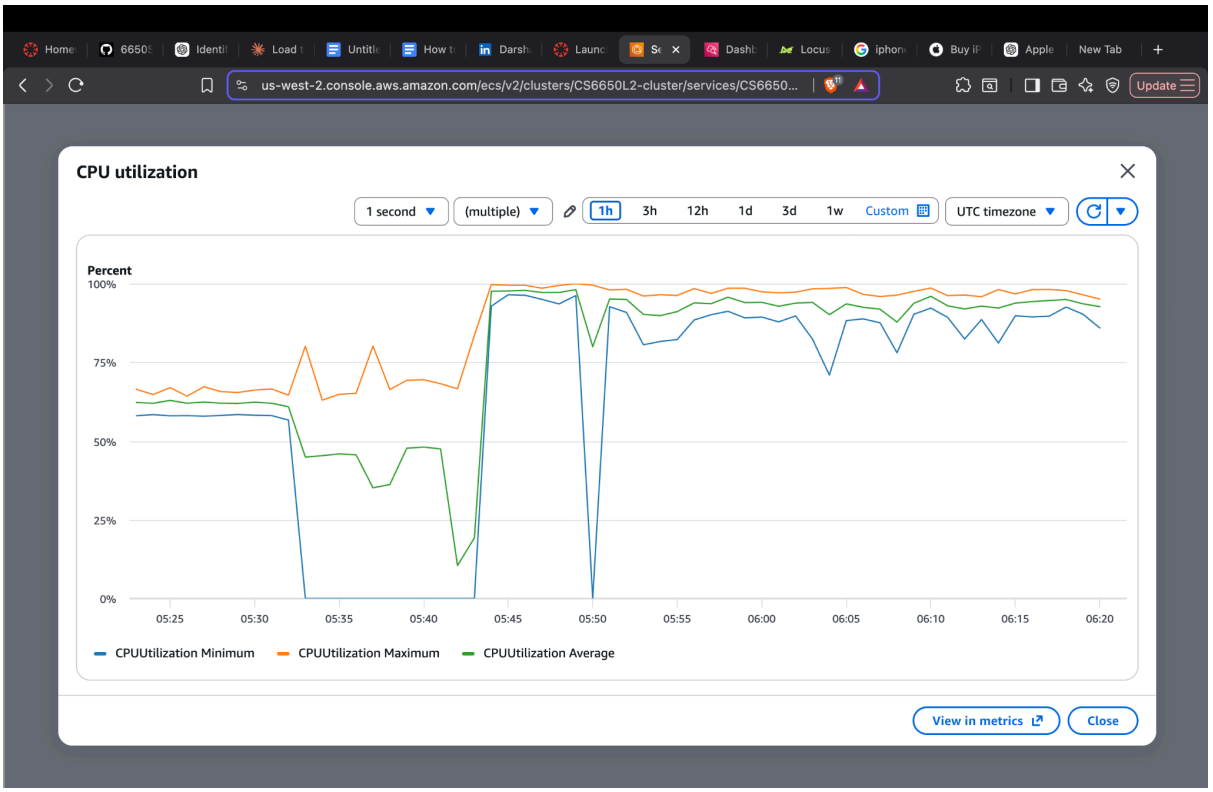


Tested with 500 users where the LB created the maximum tasks it could that is 4

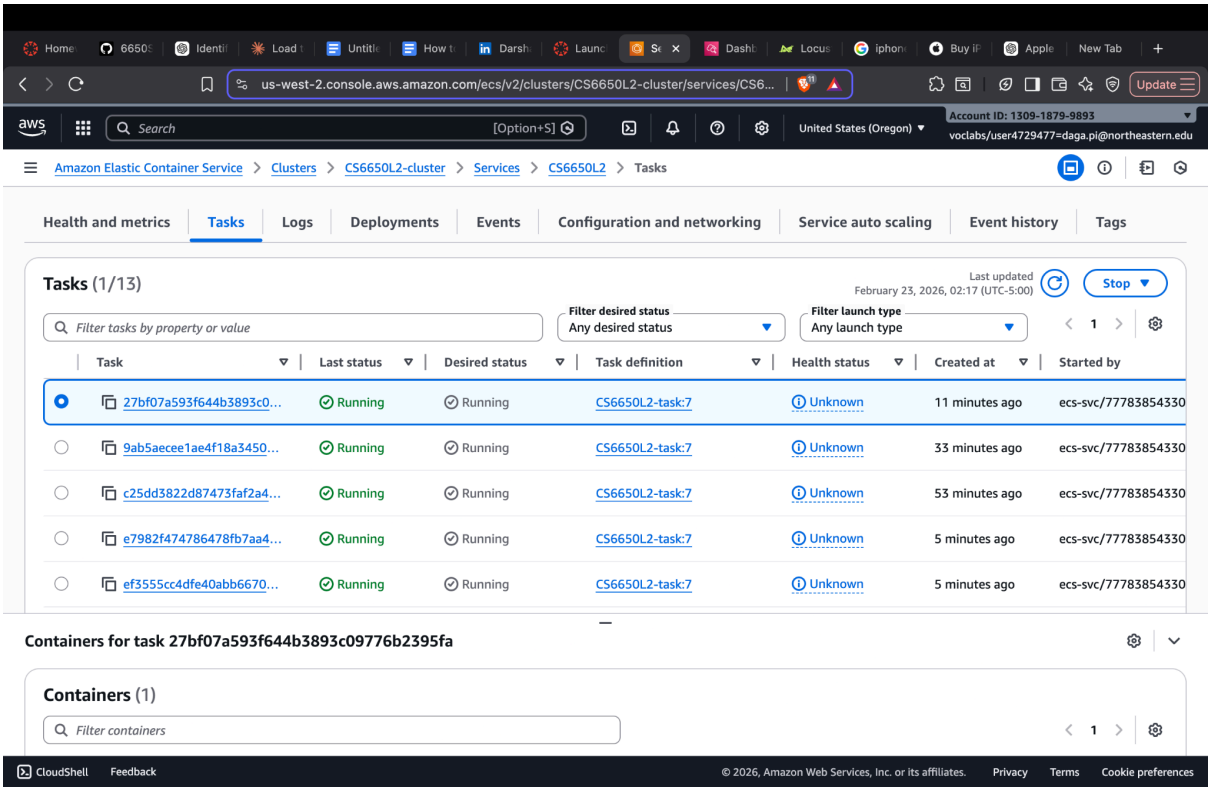

us-west-2.console.aws.amazon.com/ecs/v2/clusters/CS6650L2-cluster/services/CS6650...
Search
[Option+S]
United States (Oregon)
Account ID: 1309-1879-9893
voclabs/user4729477=daga.pi@northeastern.edu
Amazon Elastic Container Service
Clusters
CS6650L2-cluster
Services
CS6650L2
Tasks
Tasks (1/10)
Last updated
February 23, 2026, 01:25 (UTC-5:00)
Stop
Filter tasks by property or value
Filter desired status
Any desired status
Filter launch type
Any launch type
Task
Last status
Desired status
Task definition
Health status
Created at
Started by
02ed1a4eb64b491c91dd...
Running
Running
CS6650L2-task:7
Unknown
1 minute ago
ecs-svc/77783854330
74be360dcf7d4fec9e680...
Running
Running
CS6650L2-task:7
Unknown
1 minute ago
ecs-svc/77783854330
80cbc8c1862a409995ab6...
Running
Running
CS6650L2-task:7
Unknown
1 minute ago
ecs-svc/77783854330
c25dd3822d87473faf2a4...
Running
Running
CS6650L2-task:7
Unknown
1 minute ago
ecs-svc/77783854330
44257fcf85c74d18b8426...
Deactivating
Stopped
CS6650L2-task:7
Unknown
48 minutes ago
ecs-svc/77783854330
5571d52f0b46469a818d...
Stopped
Stopped
CS6650L2-task:7
Unknown
1 hour ago
ecs-svc/77783854330
Containers for task 44257fcf85c74d18b84264da86a8caa3
Containers (1)
Filter containers
https://us-west-2.console.aws.amazon.com/ecs/v2/clusters/CS6650L2-cluster/services/CS6650L2/tasks?region=us-west-2
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


localhost:8089/?tab=stats
LOCUST
Host
http://CS6650L2-alb-1967309370.us-west-2.elb.ama...
Status
RUNNING
Users
500
RPS
3874.1
Failures
1%
EDIT
STOP
RESET
STATISTICS
CHARTS
FAILURES
EXCEPTIONS
CURRENT RATIO
DOWNLOAD DATA
LOGS
LOCUST CLOUD
Type
Name
Requests
Fails
Median (ms)
95%ile (ms)
99%ile (ms)
Average (ms)
Min (ms)
Max (ms)
Average size (bytes)
Current RPS
Current Failures/s
GET
/products/search?q=alpha
3266939
37397
100
190
290
116.37
76
2583
2662.99
1173.3
0
GET
/products/search?q=beta
2175213
24825
100
190
290
116.39
76
2554
2698.67
757.8
0
GET
/products/search?q=electronics
3265724
37103
100
190
290
116.42
77
2558
2663.21
1163.8
0
GET
/products/search?q=home
1087702
12358
100
190
290
116.34
77
2384
2699.79
387.1
0
GET
/products/search?q=sports
1086352
12431
100
190
290
116.36
78
2557
2752.96
392.1
0
Aggregated
10881930
124114
100
190
290
116.38
76
2583
2682.85
3874.1
0
ABOUT

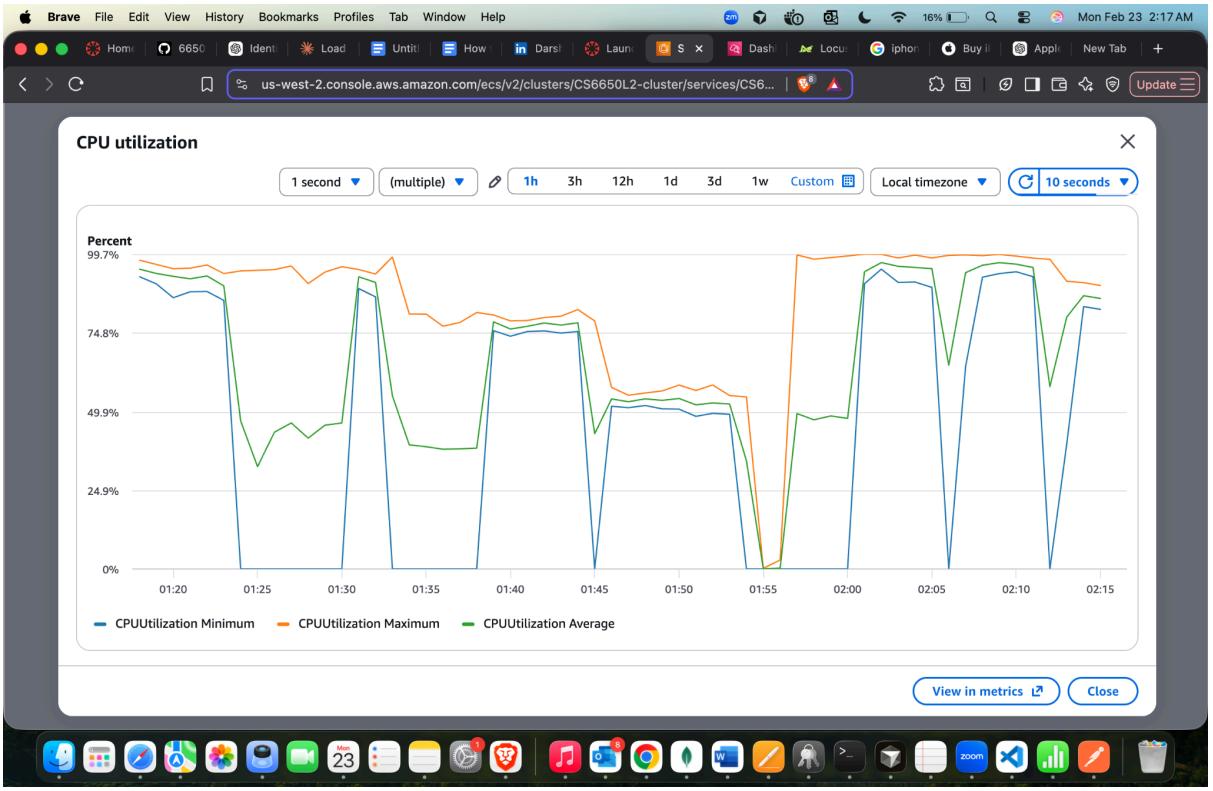
Ran a test for longer time > 5min



Ran a test with increasing the



max_capacity to 6



Slowly as tasks were adding the CPU utilization was decreasing we can evident from the graph